FML · Unit 9 · Beginner edition

Unsupervised Learning Step by Step

Group, compress and clean data when there is no answer key

Raw data → Groups → Few factors → Clean matrix

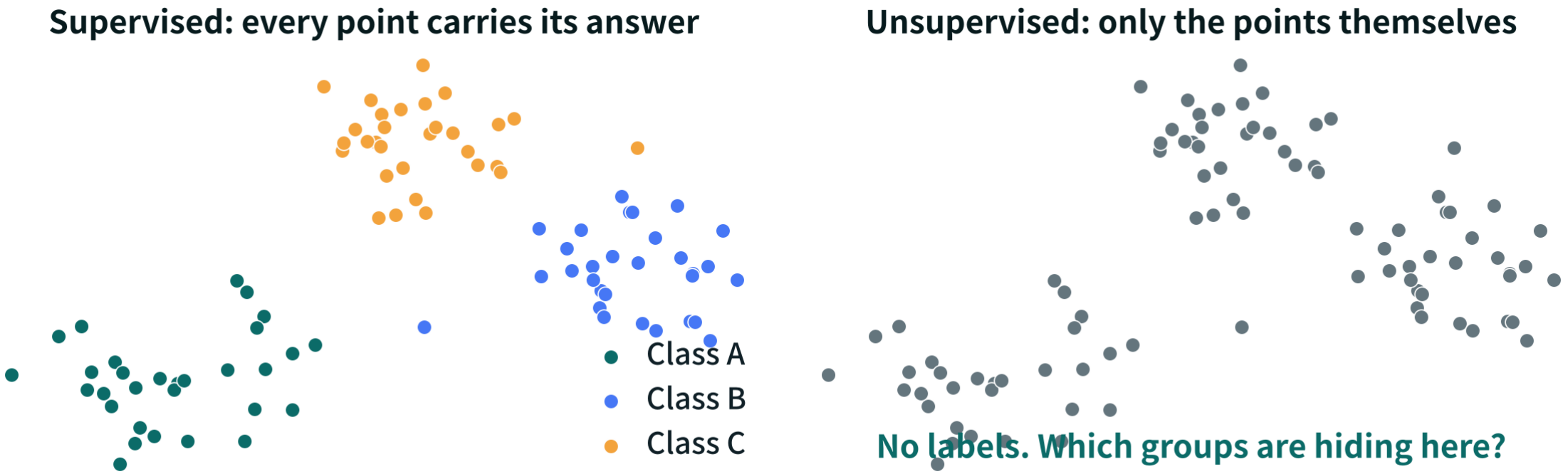
Three jobs, each built up one picture at a time

Every method is drawn from a blank start, one step per slide

01	**Finding Groups**	**Group the rows**	K-Means · Hierarchical · DBSCAN
02	**Shrinking Features**	**Squeeze the columns**	Curse of dimensionality · PCA · t-SNE
03	**Cleaning Noise**	**Fix the correlation matrix**	Fake links · Trees and grass
04	**Build It with Claude Code**	**Four prompts, every figure**	Where · Input · Output · Feedback

Unsupervised learning works without an answer key

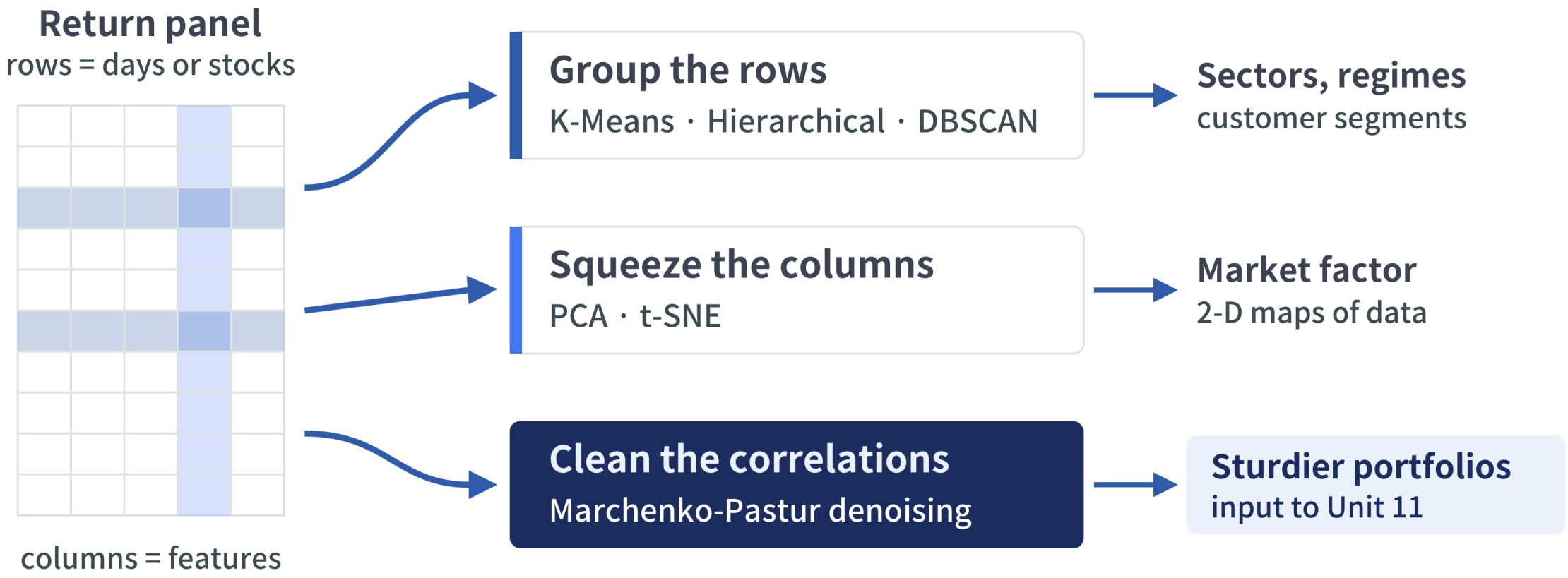
Same points, two situations: with labels (left) and without labels (right)



※ No accuracy score exists here. A result is judged by whether the structure makes sense and helps.

Each tool works on a different part of the data

Rows are grouped, columns are compressed, and the correlation matrix is cleaned



01

Finding Groups

K-Means · Hierarchical clustering · DBSCAN

K-Means finds K meeting points and sends everyone to the nearest one

Everyday picture: placing K bus stops so that passengers walk as little as possible

WHAT IT DOES

Splits into K groups

Each group has a center

EVERYDAY PICTURE

Bus stops

Walk to the nearest

YOU CHOOSE

K, the group count

Pick it with the elbow

Only two moves, repeated: assign, then update.

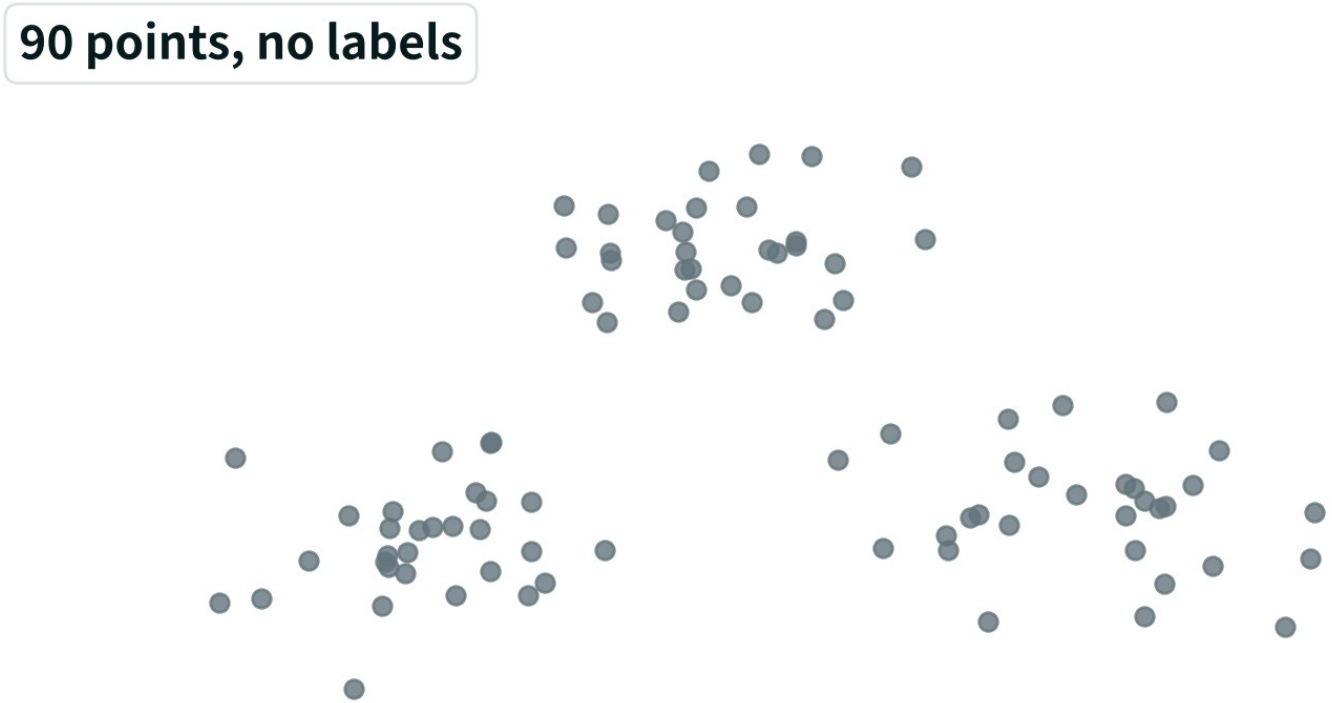
We start with points that carry no labels

90 points described by two features; we will ask for K = 3 groups

Step 1 · The data

- Each dot is one item
- Grey = no answers
- We pick K = 3

Goal: find 3 groups



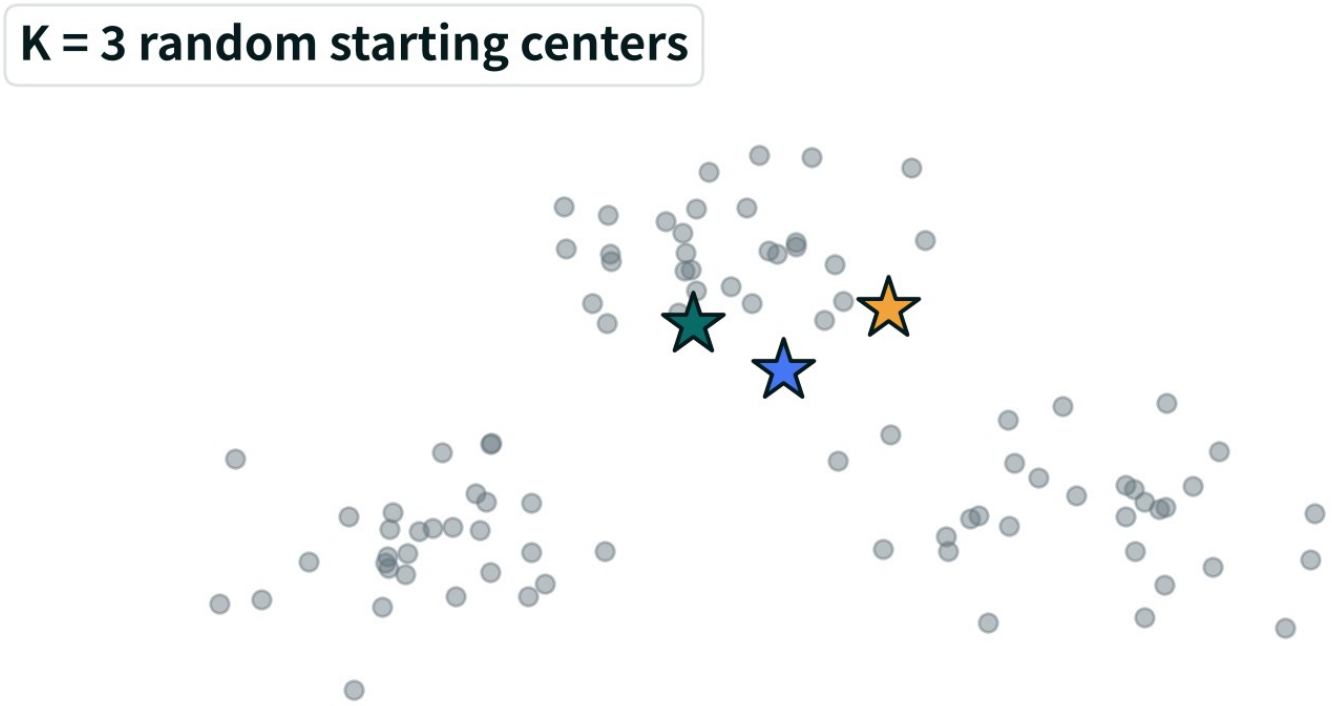
Three starting centers are dropped at random

A center (centroid) is the meeting point of one group

Step 2 · Start

- Stars are centers
- Bad spots on purpose
- The loop will fix them

Start can be anywhere



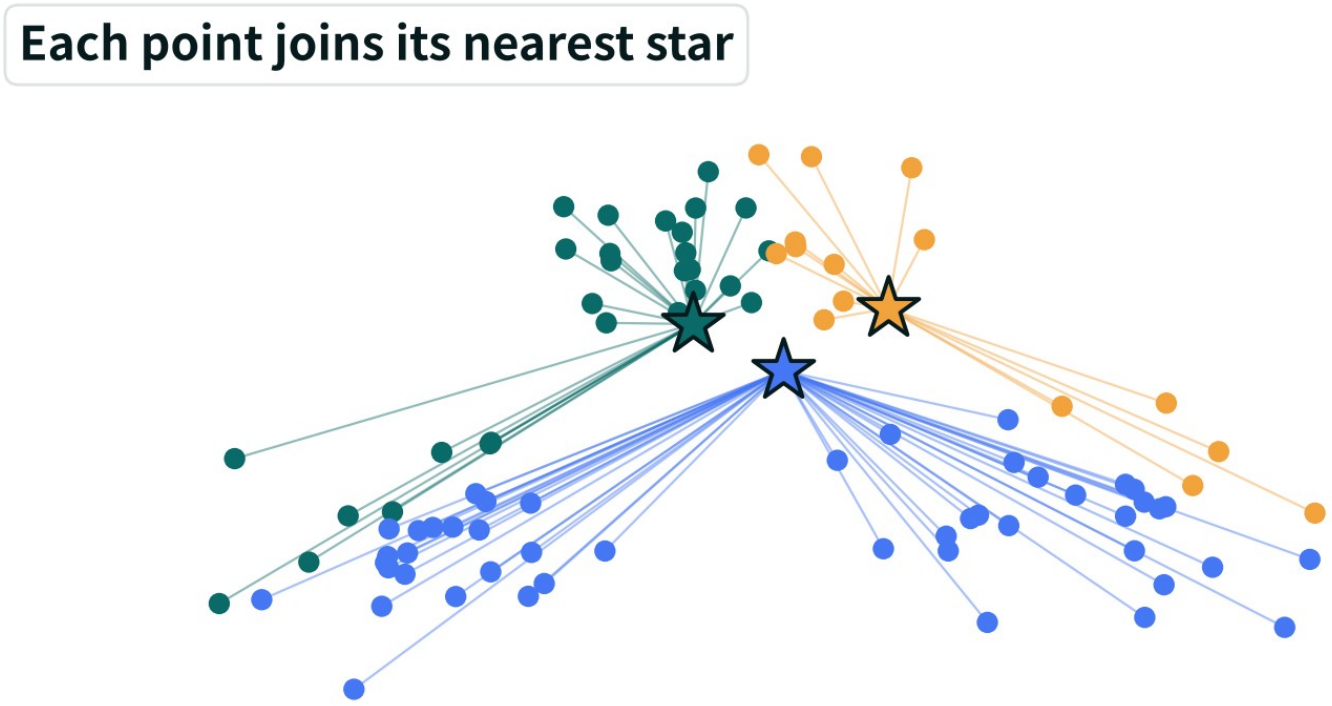
Every point joins the center nearest to it

Distance is measured as a straight line on the chart

Step 3 · Assign

- Measure to each star
- Take nearest colour
- Lines: who joined

Move 1: assign



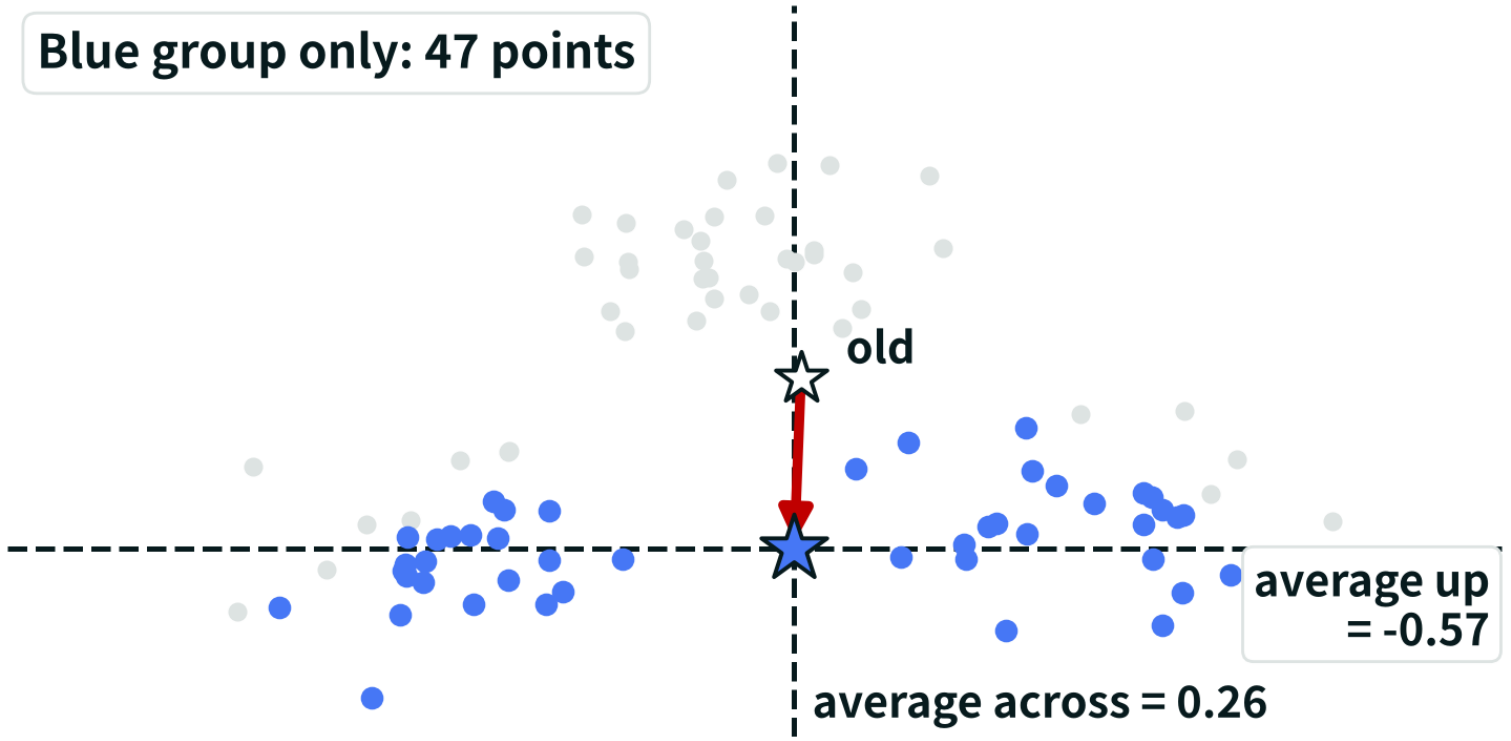
Zoom in: the new center is just the group's average

Blue group only: average its left-right positions, then its up-down positions

Step 4 · One group

- Average across
- Average up
- Where lines cross

New star = average



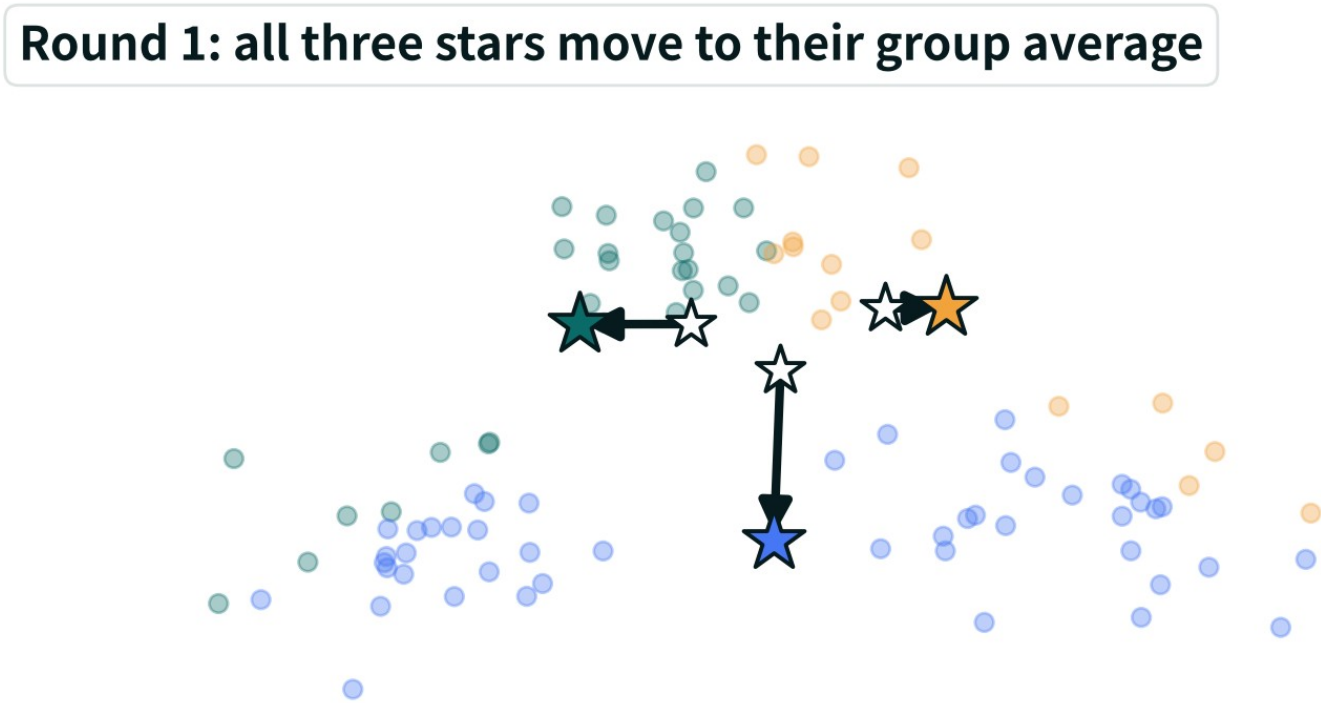
Round 1: all three centers move to their group average

Hollow star = old spot, filled star = new spot

Step 5 · Update all

- Same rule ×3
- Arrow = the move
- Big jumps at first

Move 2: update



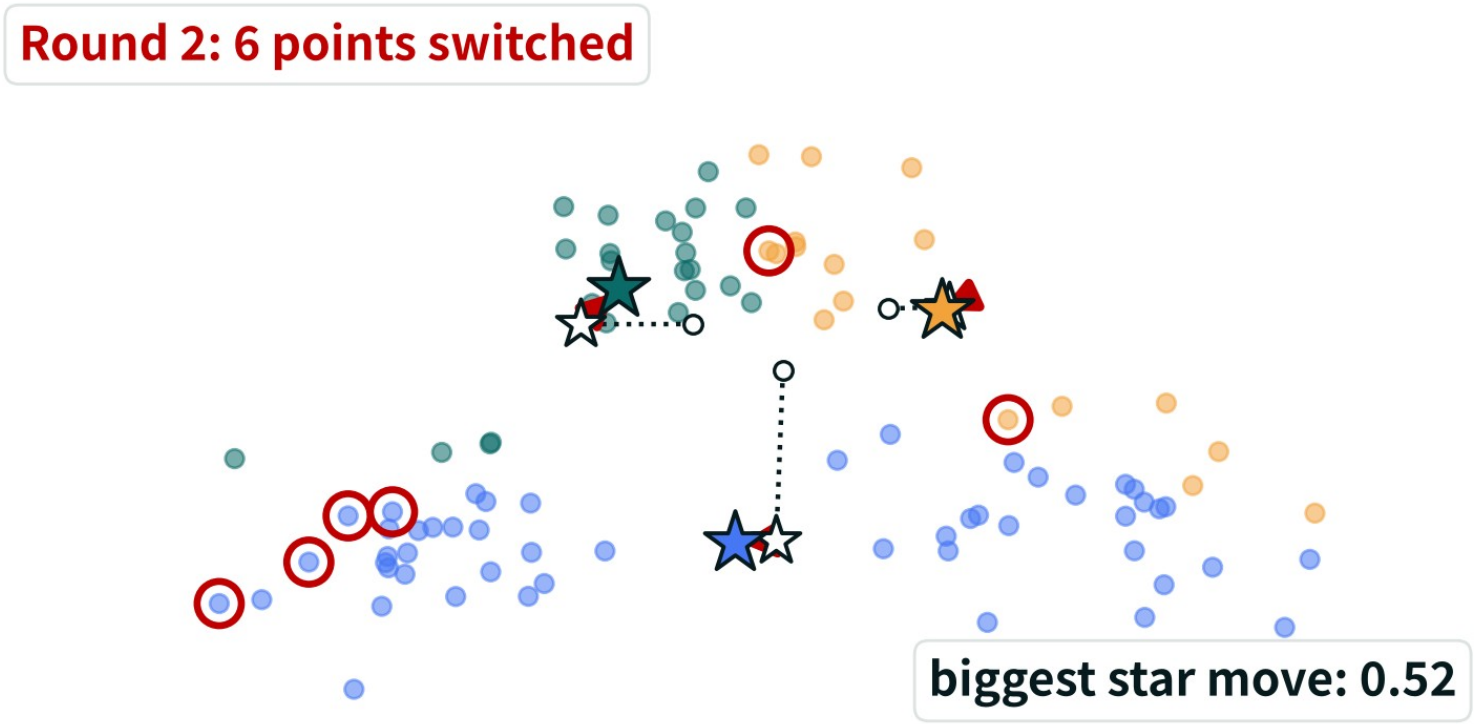
Round 2: six points now sit closer to another center

Assign again (red rings = switched), then move again (red arrows)

Step 6 · Round 2

- Assign again
- Red ring = switched
- Small dots = past spots

Assign, then update



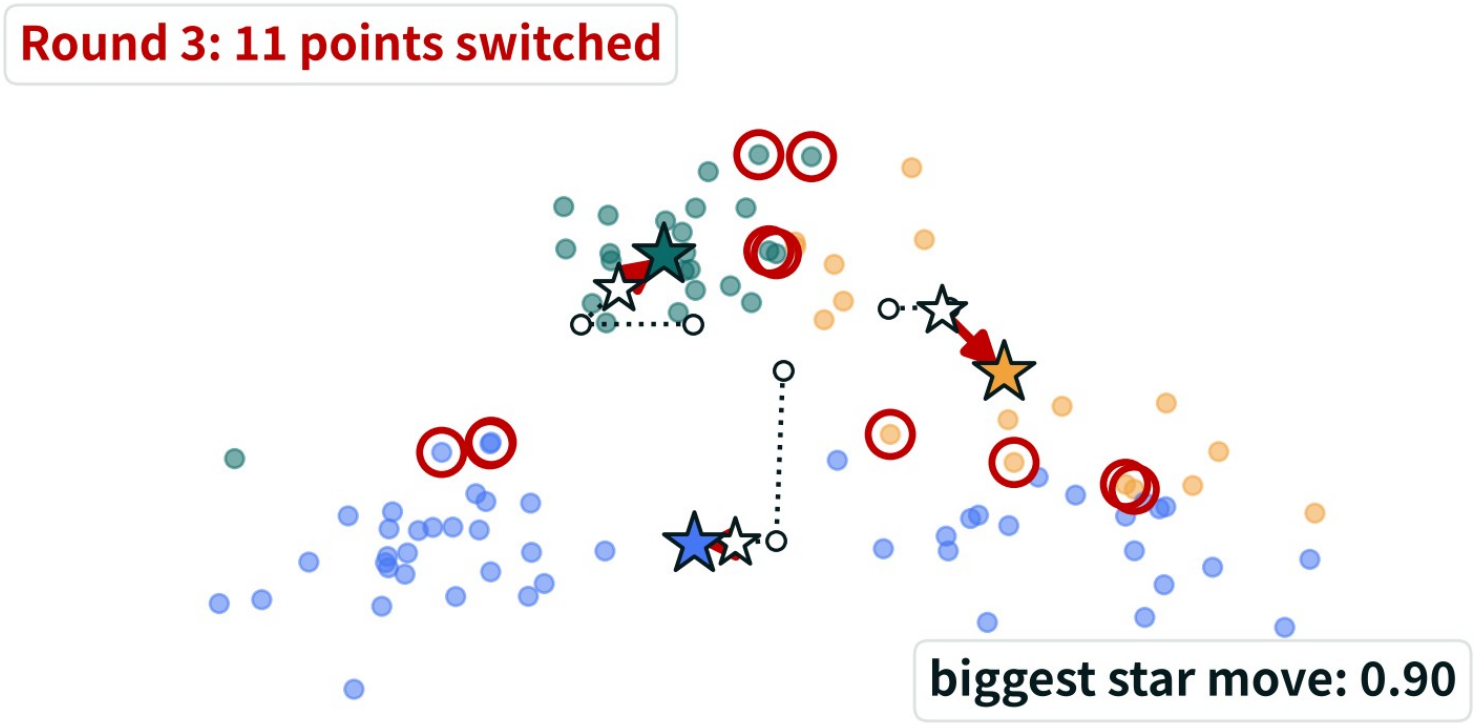
Round 3: the amber center starts sliding down-right

More points switch, so the stars drift toward the real clumps

Step 7 · Round 3

- 11 switched
- Amber heads down
- Dotted = its path

Stars follow points



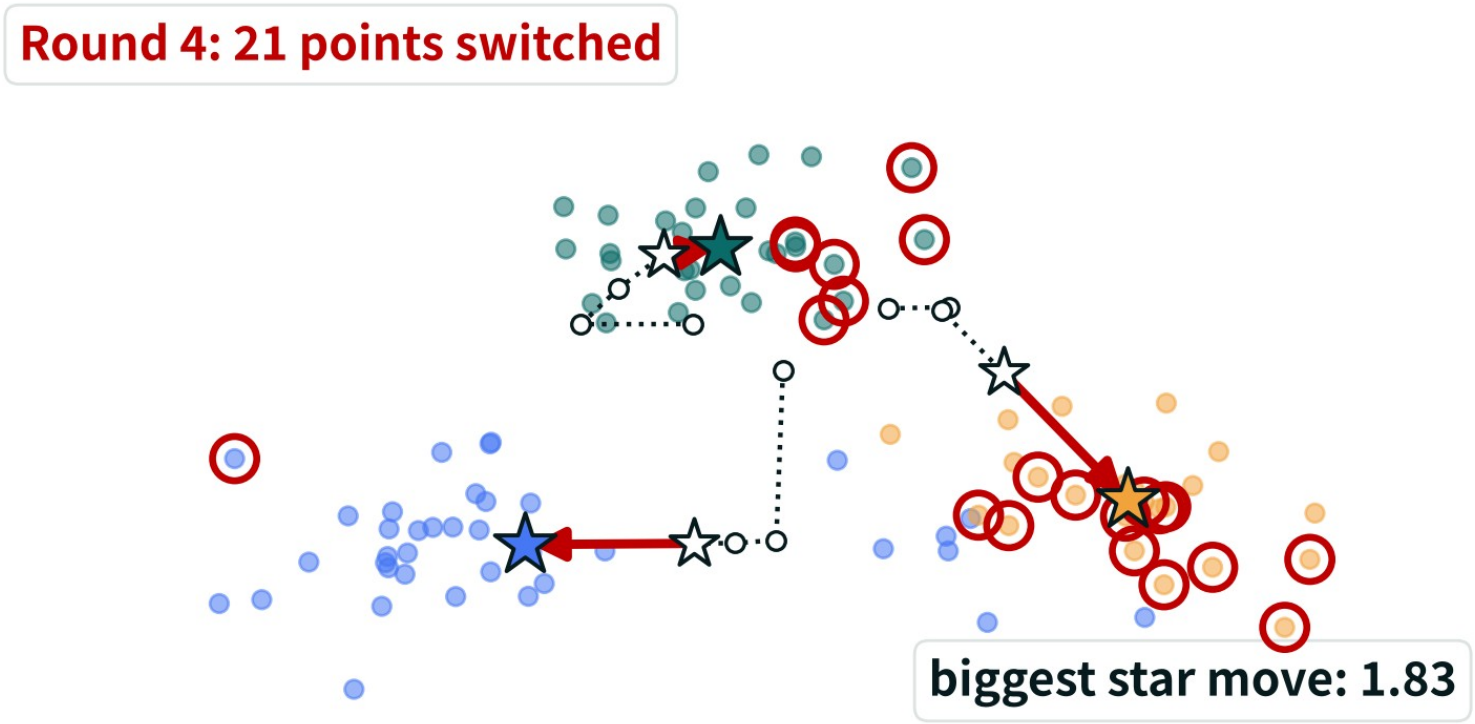
Round 4: the biggest jump, as two stars find their clumps

21 points switch; amber and blue each jump into a separate clump

Step 8 · Round 4

- 21 switched
- Longest arrows
- J drops 277 → 102

The big fix



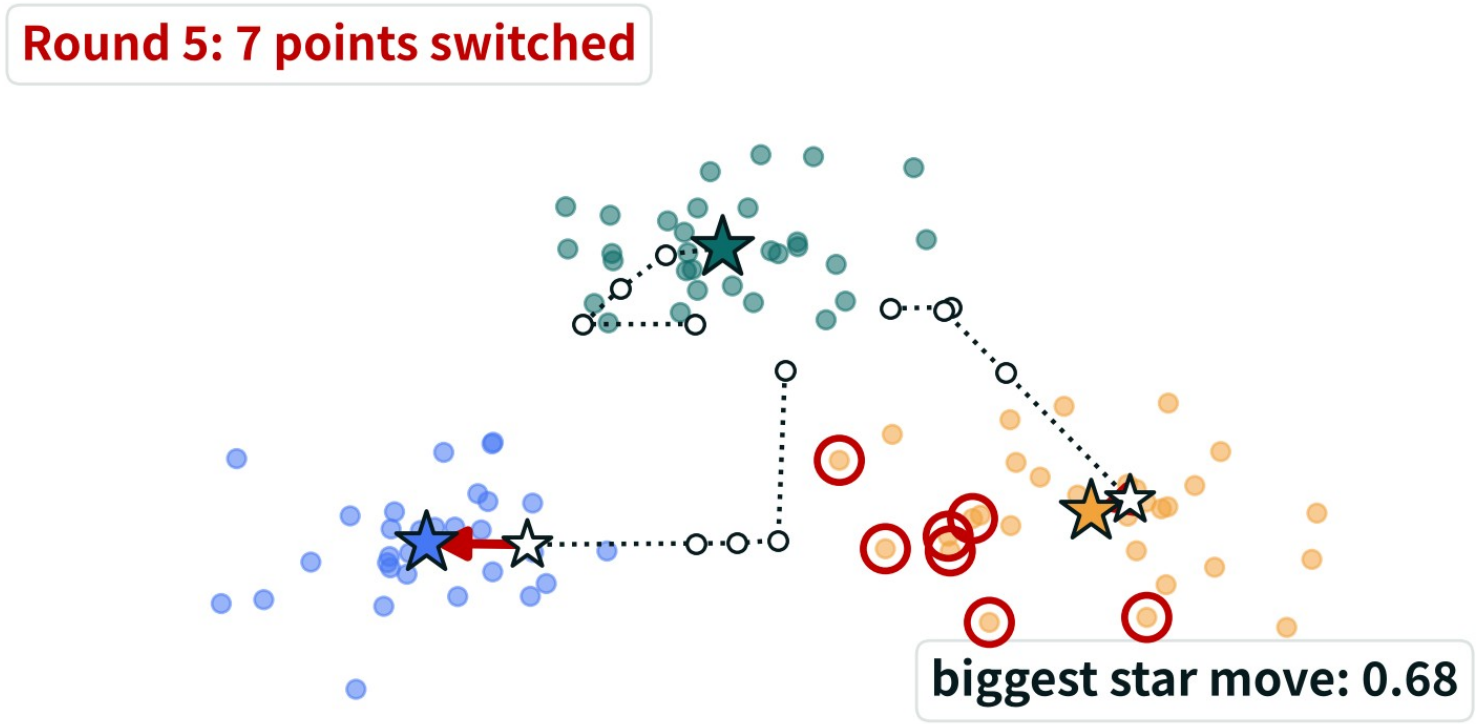
Round 5: only small touch-ups remain

A few edge points switch, and the stars shift a little

Step 9 · Round 5

- 7 switched
- Short arrows
- Almost settled

Fine tuning



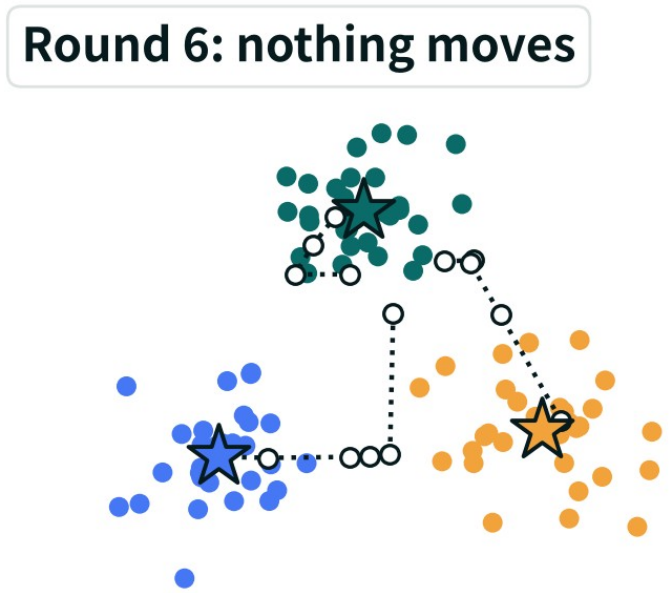
The loop stops when no point changes group

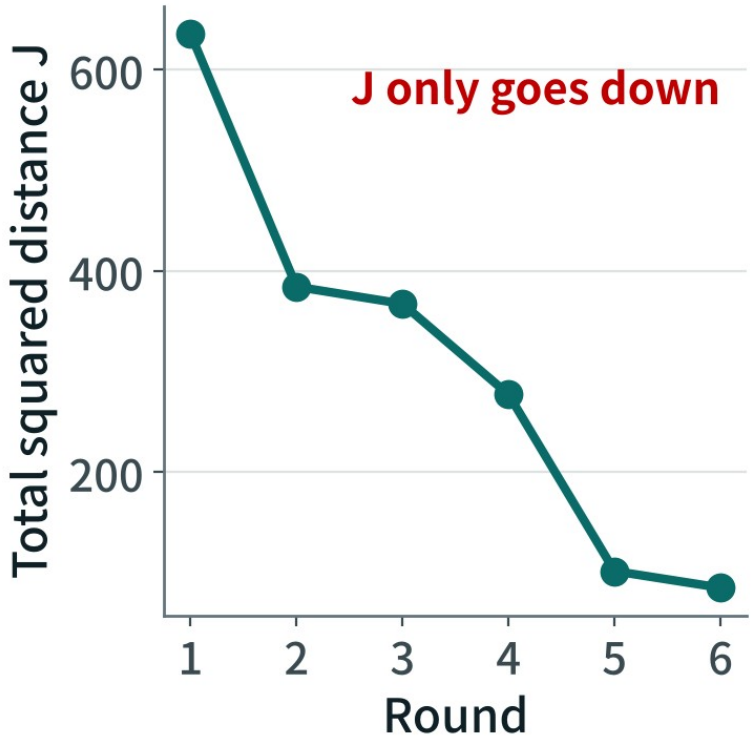
Right: total squared distance J after each round

Step 10 · Done

- Dotted = star paths
- J falls every round
- Here: 6 rounds

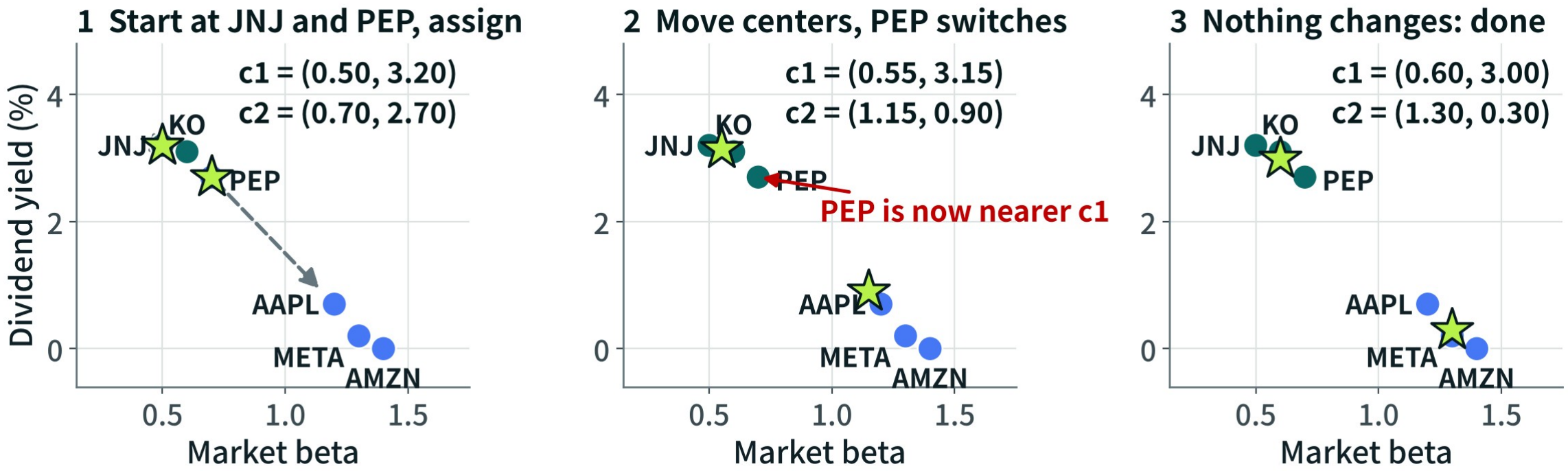
Converged





Six stocks split into defensive and growth in two features

Market beta and dividend yield (%). K = 2, starting centers placed on JNJ and PEP



※ Step 2: PEP's squared distance is 0.23 to c1 but 3.44 to c2, so it switches. Illustrative figures.

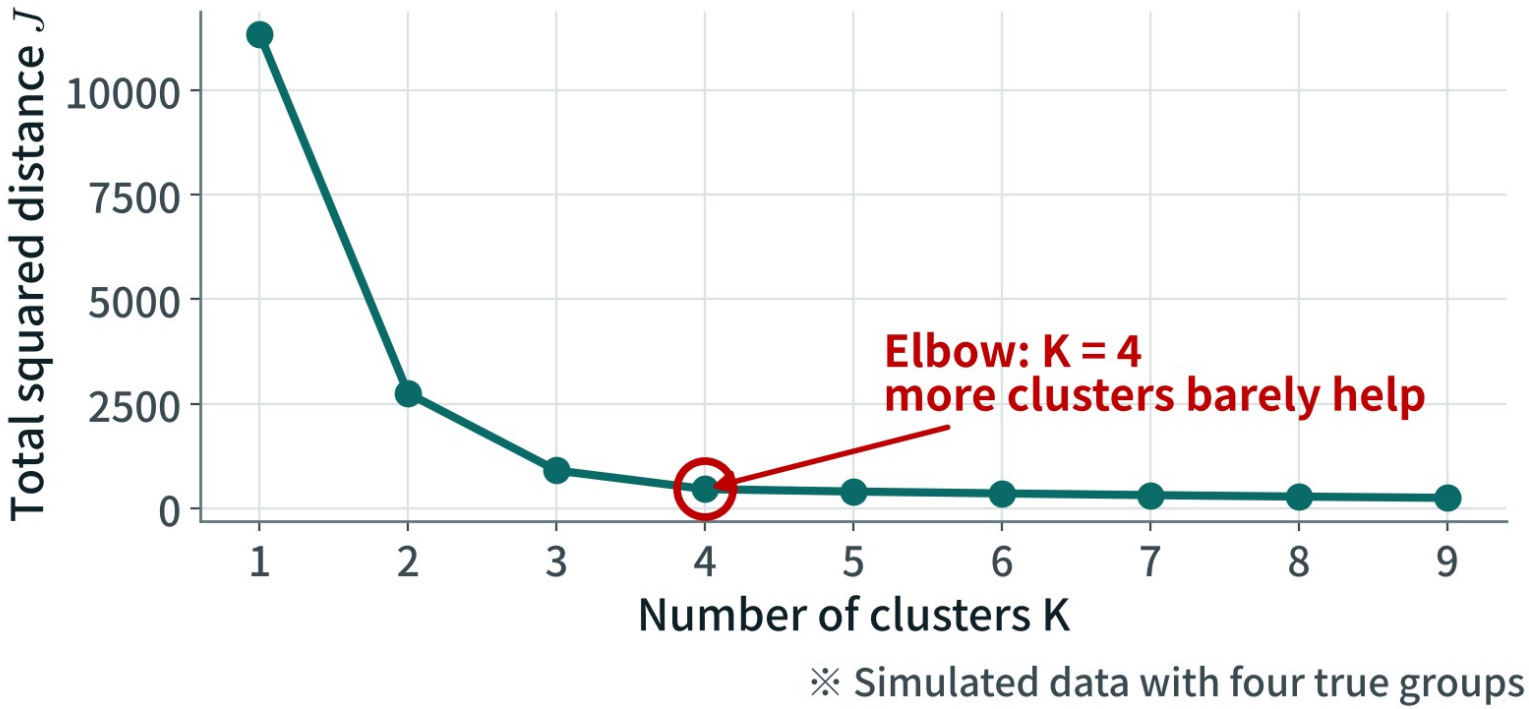
The elbow picks K where extra clusters stop paying

J always falls as K grows, so look for the bend, not the minimum

Reading the curve

- Too few K: lumped
- Too many K: split noise
- Bend: real groups

Elbow + common sense



Hierarchical clustering builds a family tree of the data

Everyday picture: close relatives join first, then families, then the whole clan

WHAT IT DOES

Merges closest groups

One pair at a time

EVERYDAY PICTURE

A family tree

Close kin join first

YOU CHOOSE

Where to cut

Sets the group count

No K up front: decide after seeing the tree.

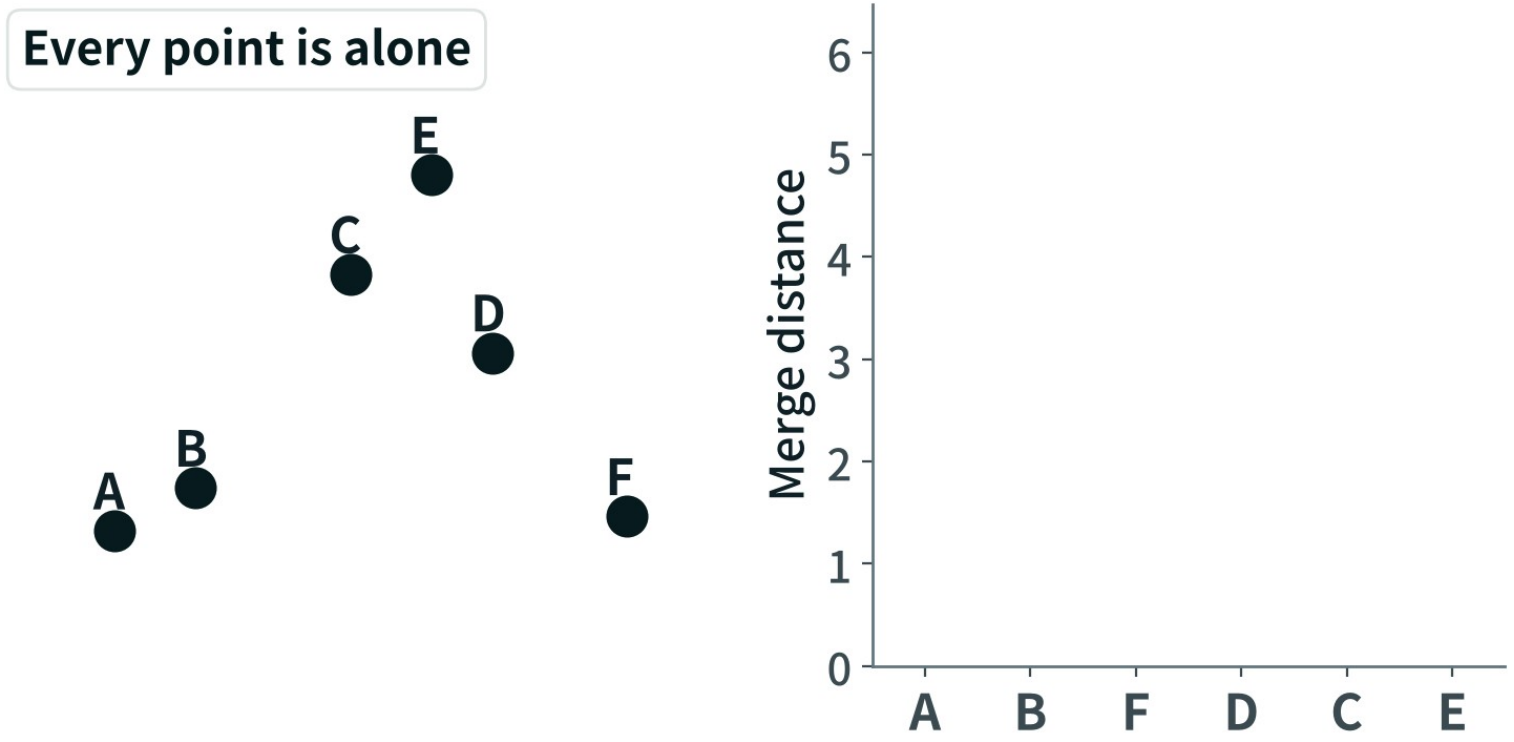
Every point starts as a group of its own

Left: six points. Right: the tree (dendrogram) we will grow

Step 1 · Start

- Six points, six groups
- The tree is empty
- Height = distance

Start: 6 groups



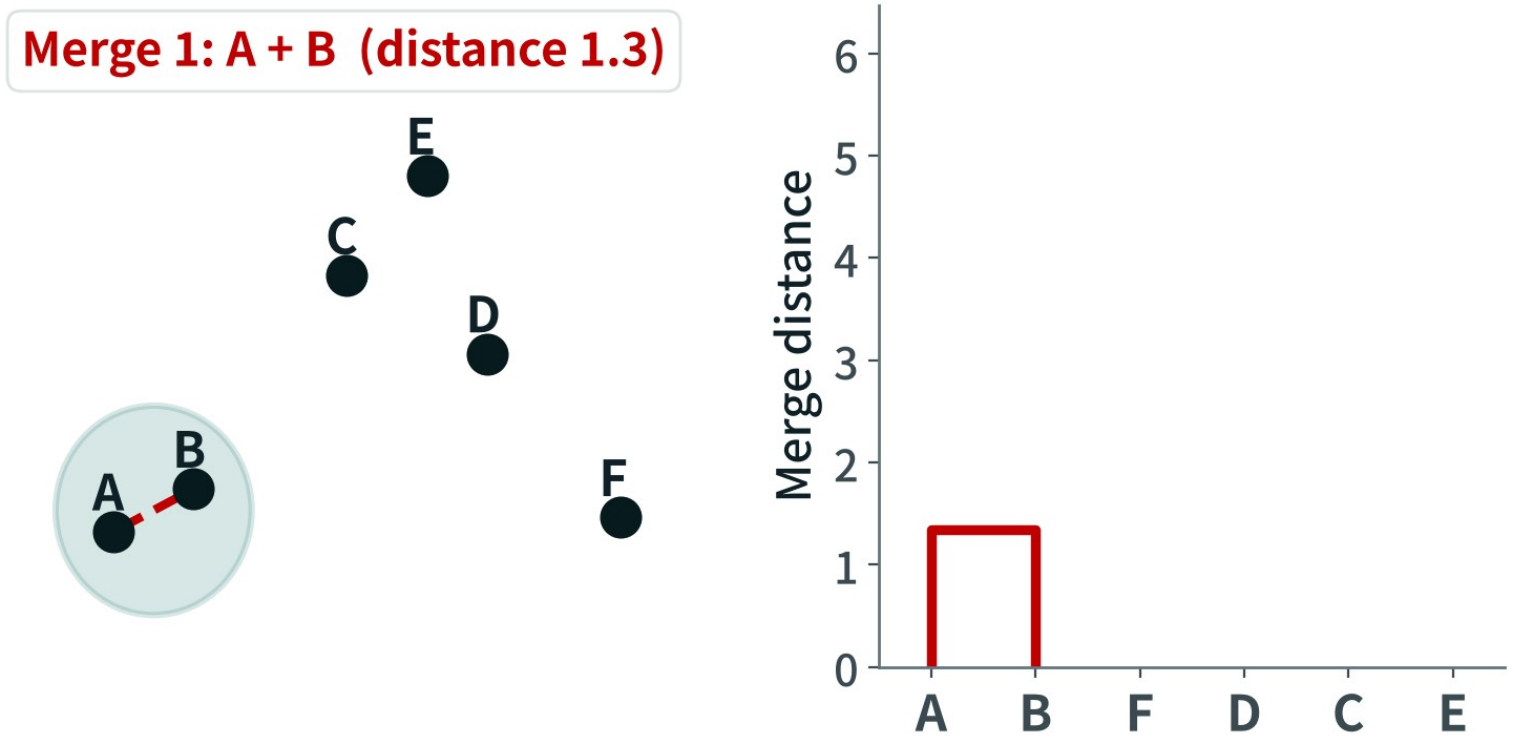
The two closest points merge first

Red dashes join the pair; a link appears in the tree at their distance

Step 2 · Merge 1

- Find the smallest gap
- Join A and B
- Draw a link at 1.3

6 → 5 groups



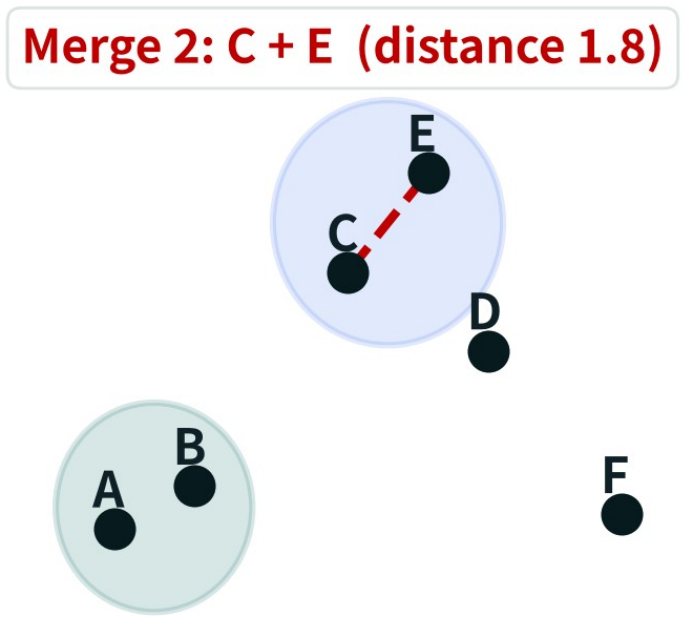
The next closest pair merges a little higher

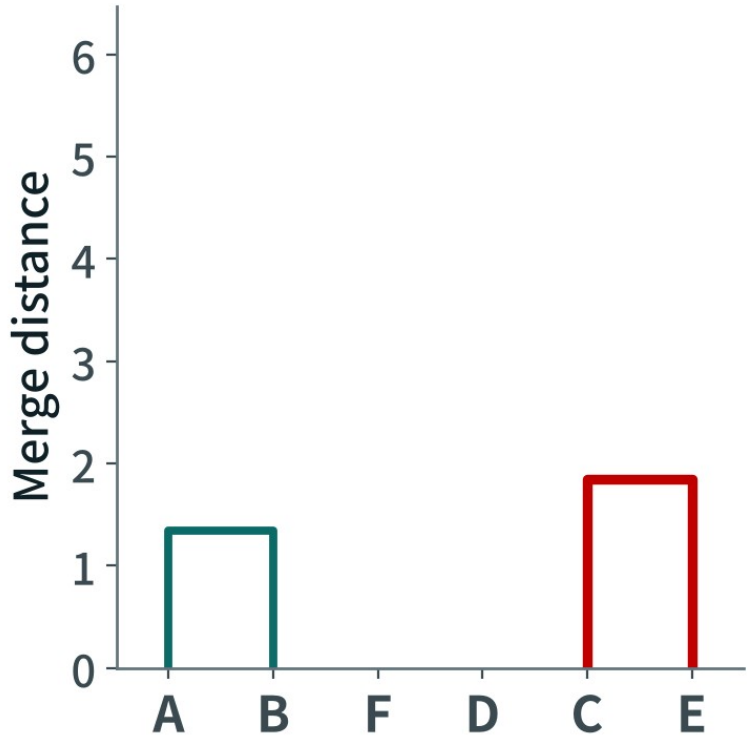
After each merge, the search starts again over all groups

Step 3 · Merge 2

- Search all pairs again
- Join C and E
- Link height = distance

5 → 4 groups





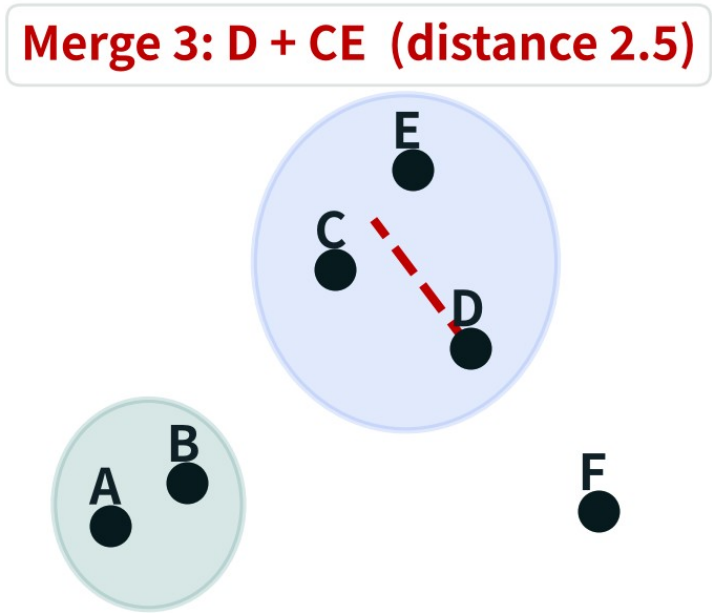
A single point can join a group that already exists

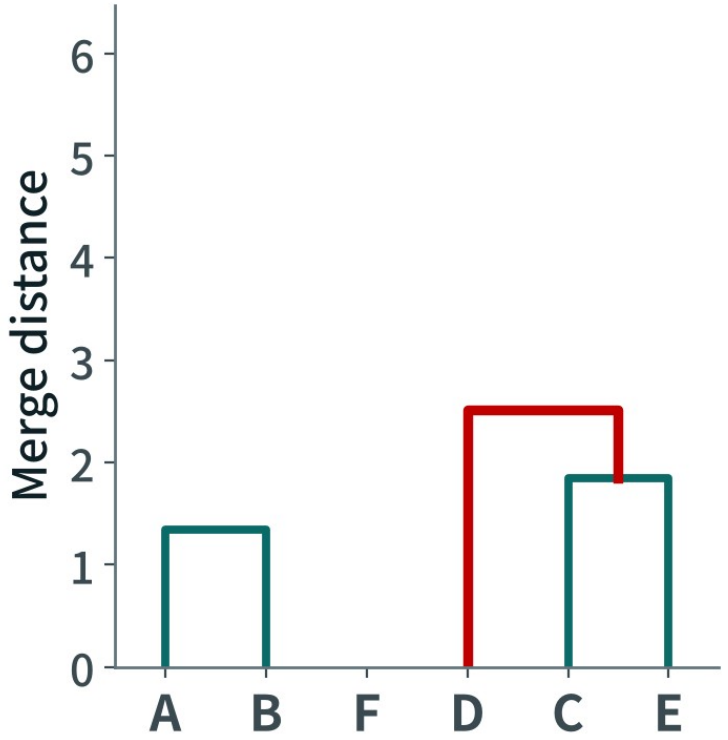
Measuring a point-to-group distance needs a rule, called linkage (two slides ahead)

Step 4 · Merge 3

- D joins C and E
- Groups act as one
- Rule: average distance

4 → 3 groups





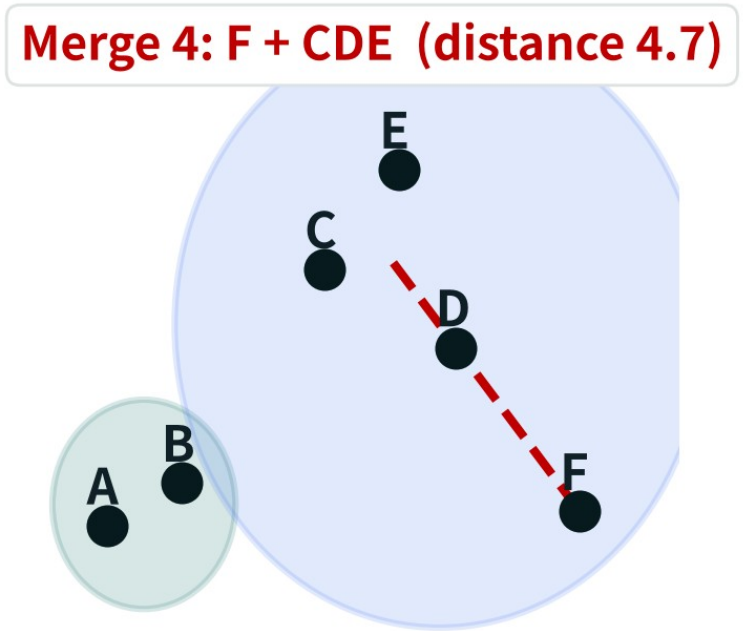
Links grow taller as more distant groups merge

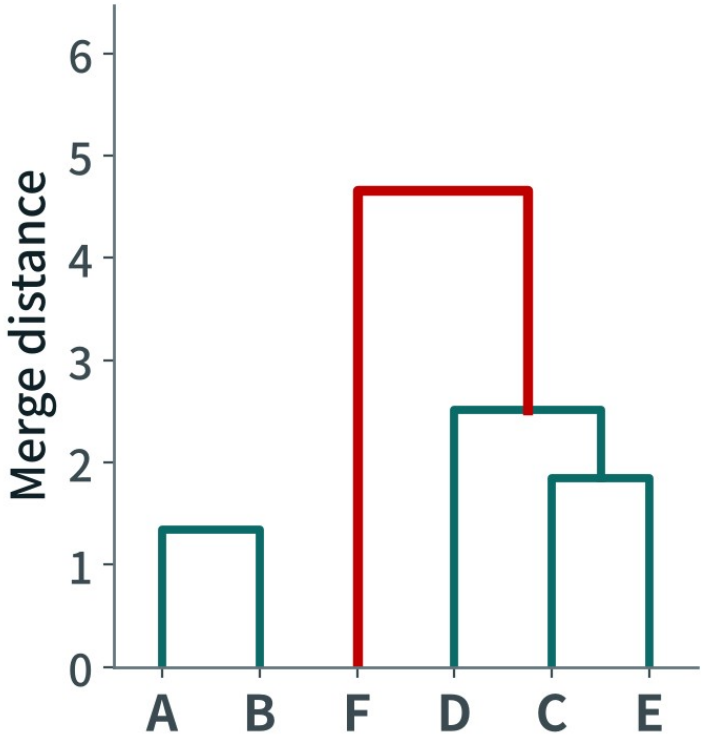
A tall link means the two groups were far apart before they joined

Step 5 · Merge 4

- F joins C, D, E
- Links get taller
- Tall jump = real gap

3 → 2 groups





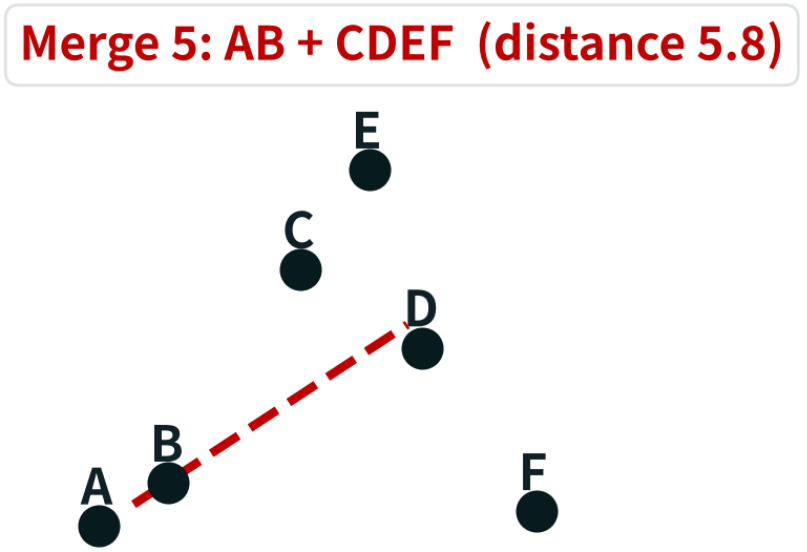
The last merge joins everything into one tree

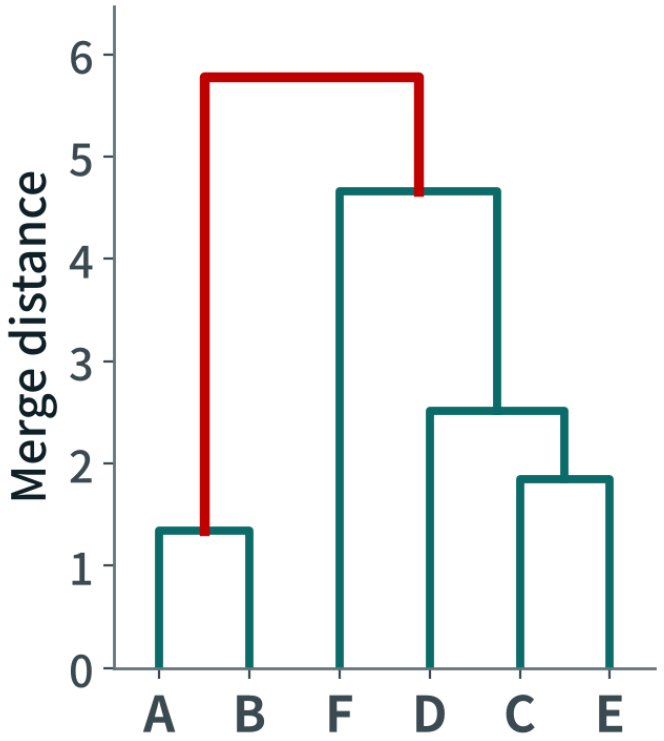
Six points need exactly five merges

Step 6 · Merge 5

- AB meets CDEF
- Height 5.8: biggest gap
- The tree is complete

2 → 1 group





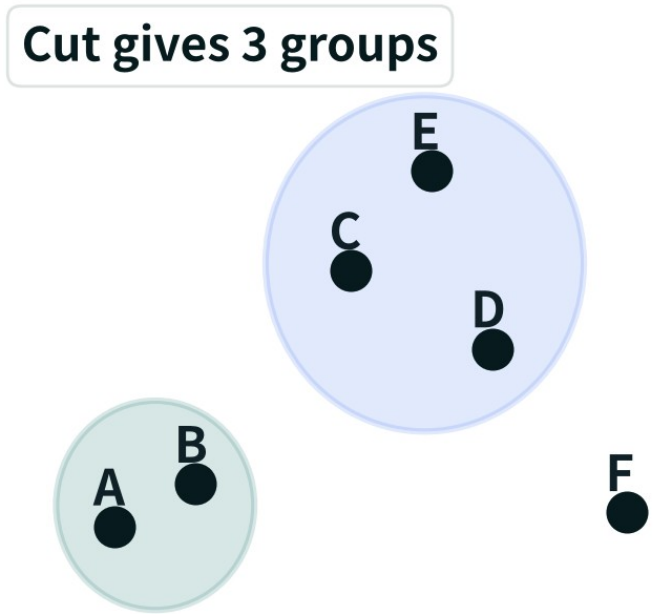
Cutting the tree at one height gives the clusters

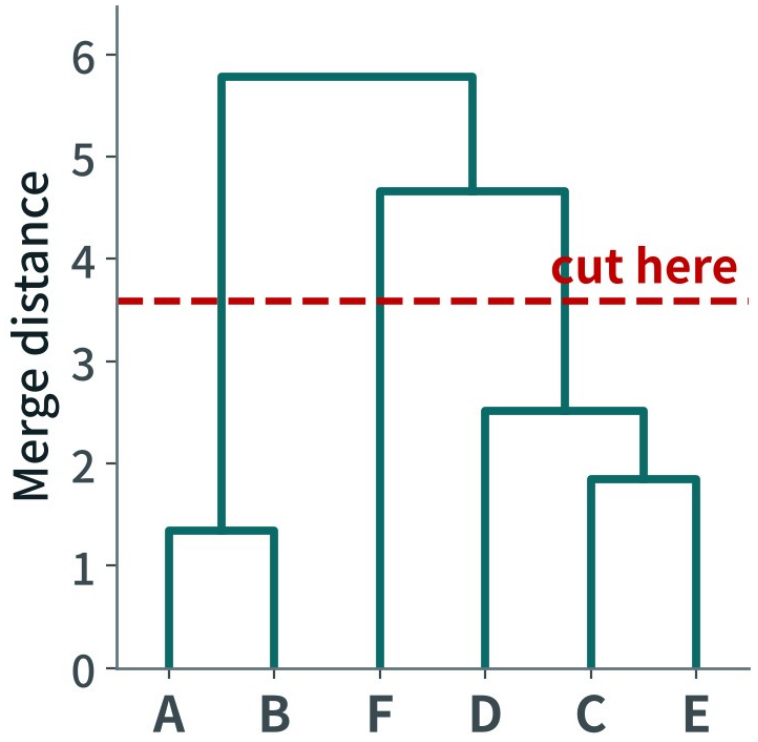
Every branch that crosses the cut line becomes one cluster

Step 7 · Cut

- Draw a level line
- Count branches cut
- Here: 3 groups

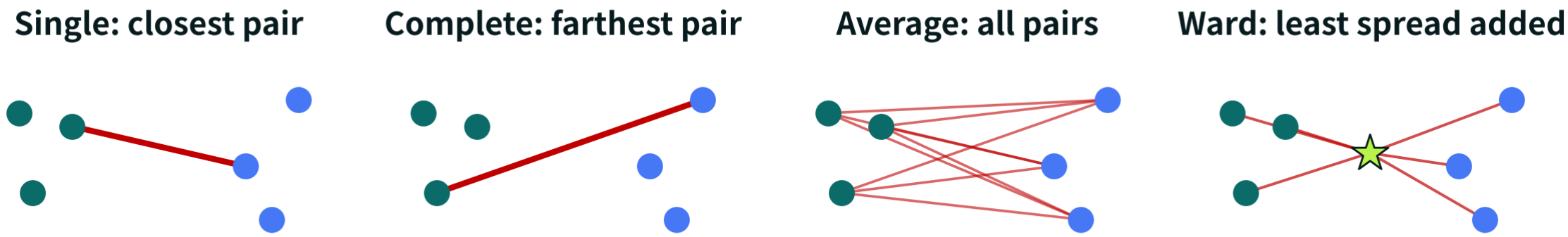
Cut higher = fewer





The linkage rule defines the distance between two groups

Points are easy to measure; groups need a rule. The red lines show what each rule measures



※ Single links long chains, Complete keeps groups compact, Ward is the usual default.

DBSCAN finds crowded regions and calls lonely points noise

Density-Based Spatial Clustering of Applications with Noise (Ester et al., 1996)

WHAT IT DOES

Groups crowded areas

Any shape works

EVERYDAY PICTURE

A playground

Arm's reach = ε

YOU CHOOSE

ε and minPts

Reach and crowd size

There is no K: the data decides how many clusters.

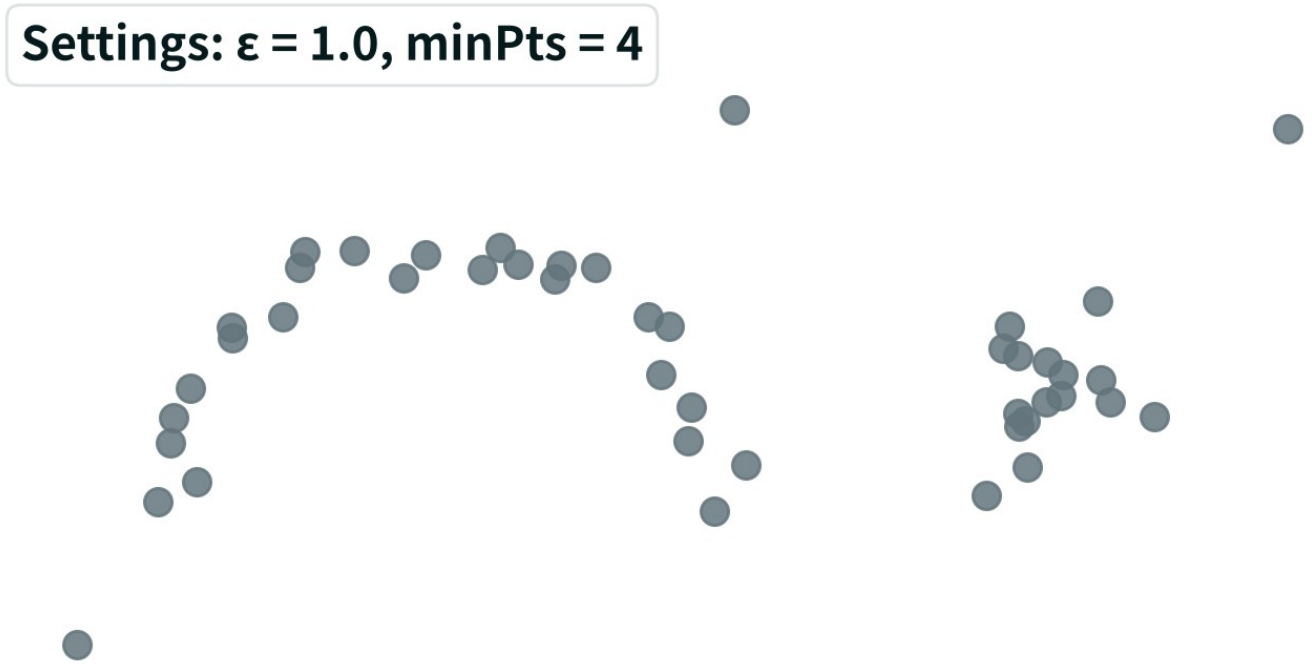
Two settings define what counts as crowded

ε = how far each point reaches; minPts = how many points make a crowd (counting itself)

Step 1 · Settings

- ε = 1.0 (reach)
- minPts = 4 (crowd)
- No K anywhere

All points start grey



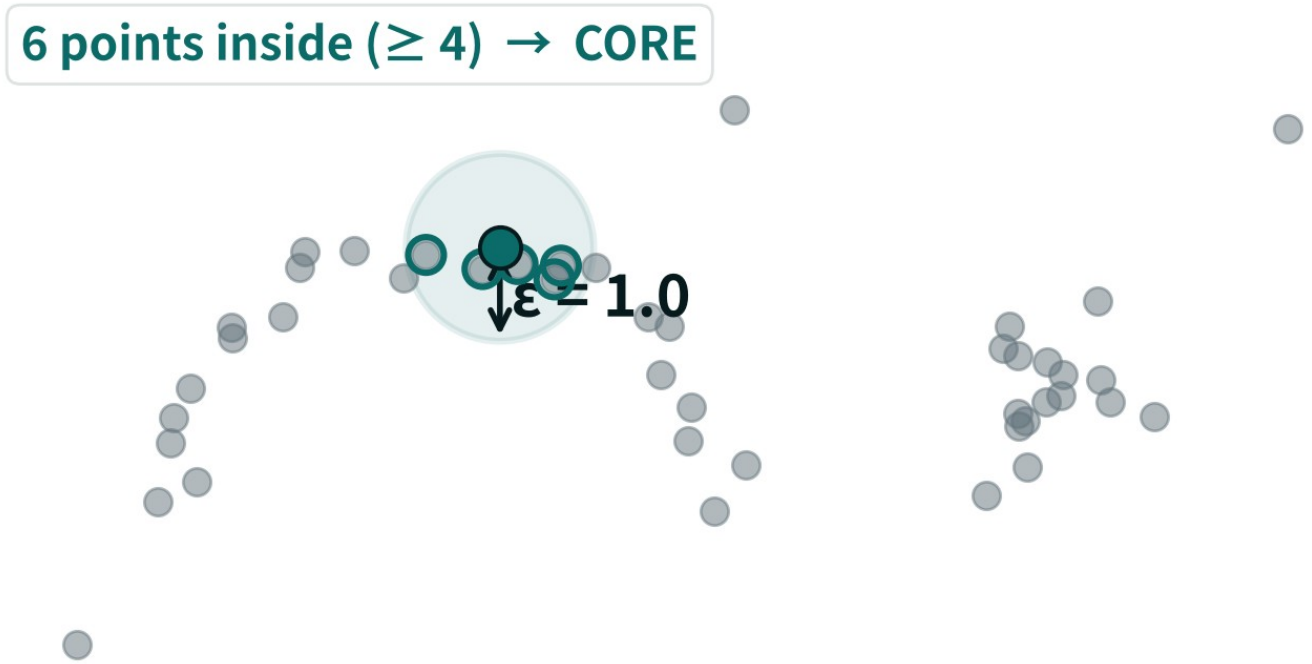
A point with enough neighbours inside its circle is a core point

Draw a circle of radius ε around the point and count who is inside

Step 2 · Core

- Circle of radius ε
- Count points inside
- 6 ≥ 4 → core

Core = in a crowd



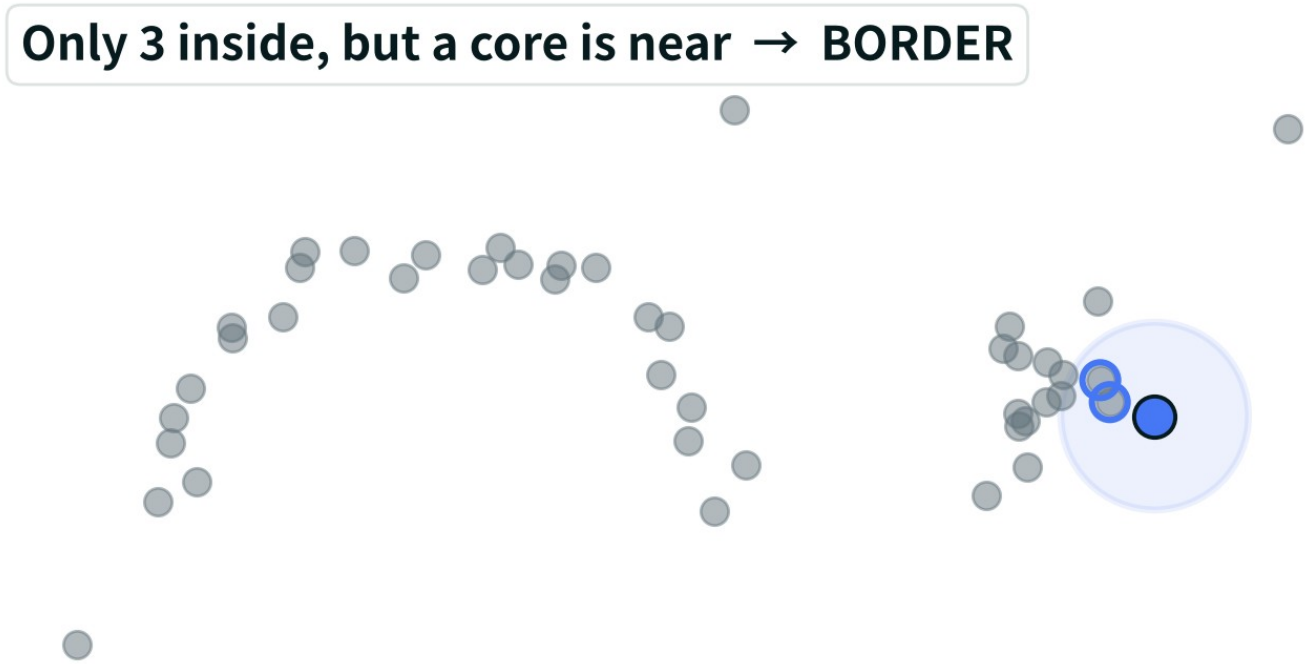
A point next to a core but not crowded itself is a border point

Its own circle is thin, but it sits inside a core point's circle

Step 3 · Border

- Few points inside
- But a core is within ε
- → border

Border = edge of a crowd



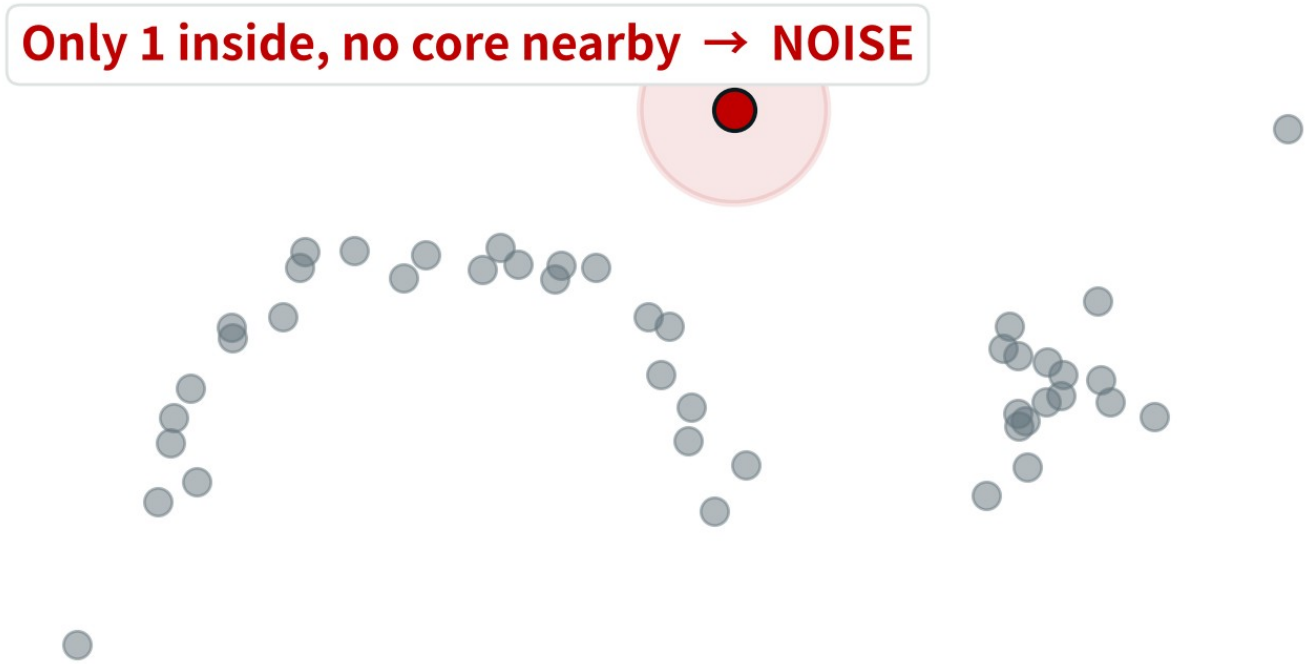
A point with no crowd and no core nearby is noise

Nobody is within ε, so it belongs to no cluster

Step 4 · Noise

- Only itself inside
- No core within ε
- → noise

Noise = a loner



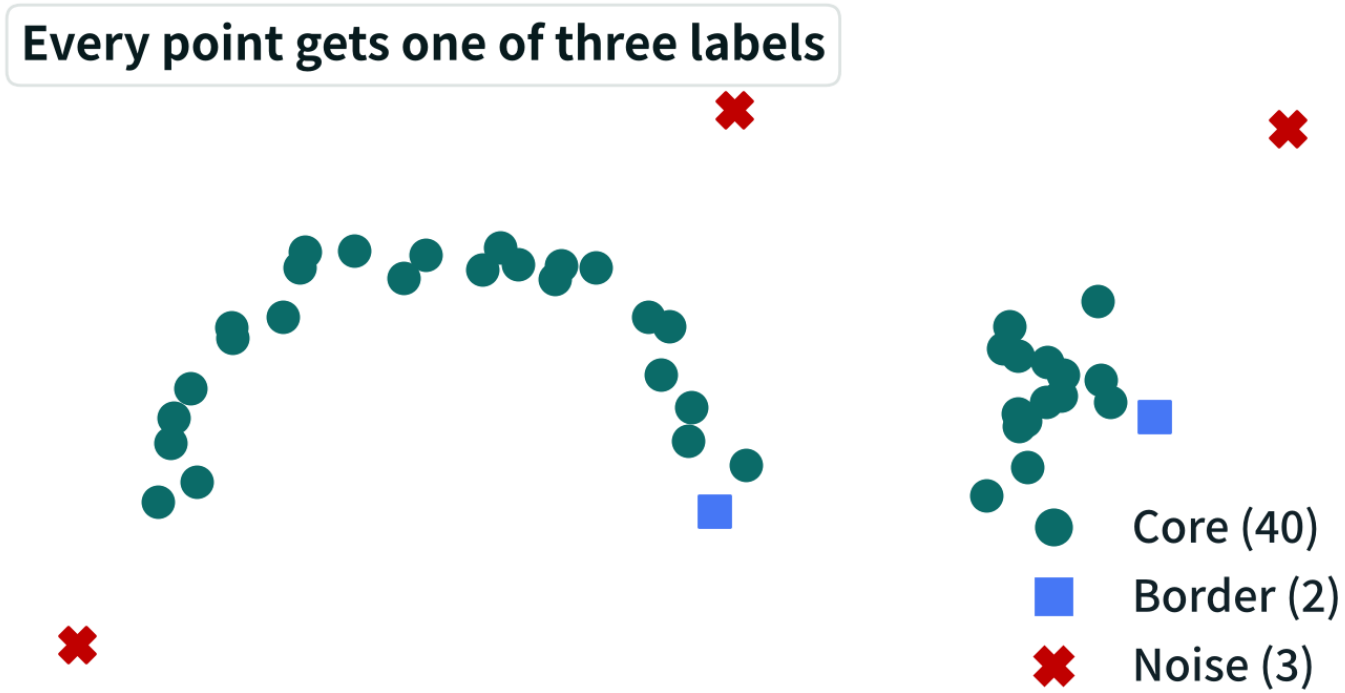
Every point receives one of the three labels

The same circle test is repeated for every point

Step 5 · Label all

- Teal dot = core
- Blue square = border
- Red X = noise

Next: connect cores



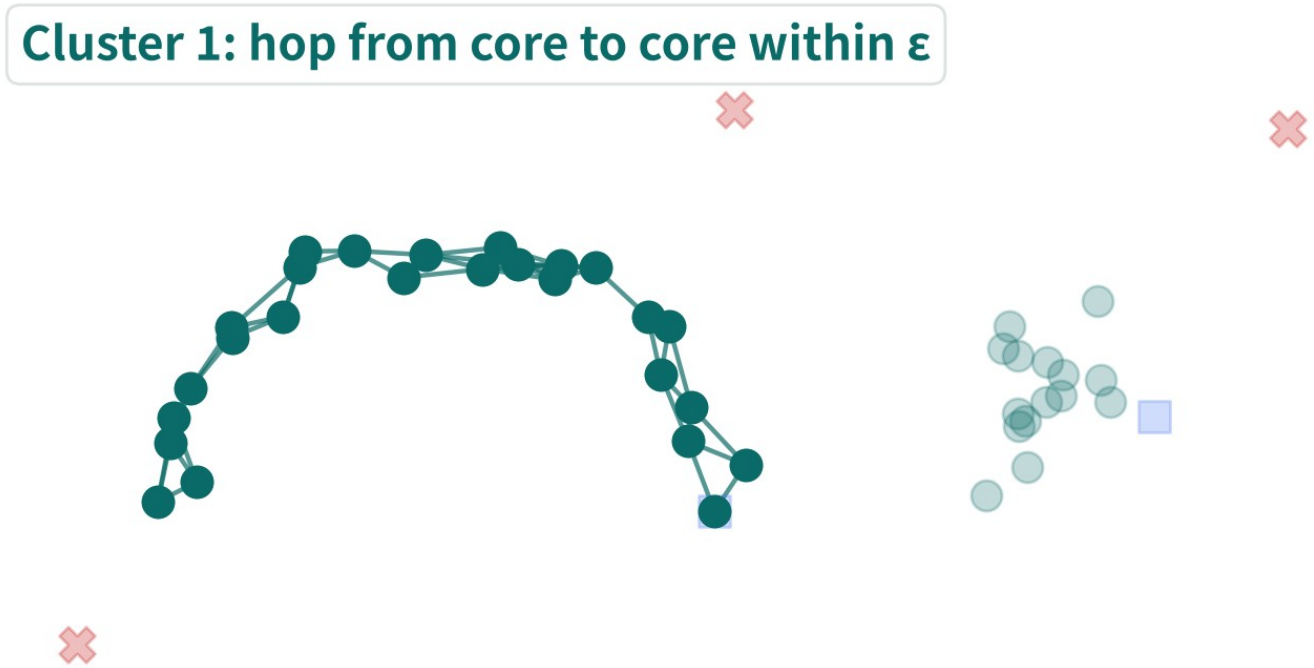
A cluster grows by hopping from core to core

Start at any core, add everyone within ε, and repeat from each new core

Step 6 · Grow

- Start at one core
- Add all within ε
- Hop on from new cores

Stops at the edges



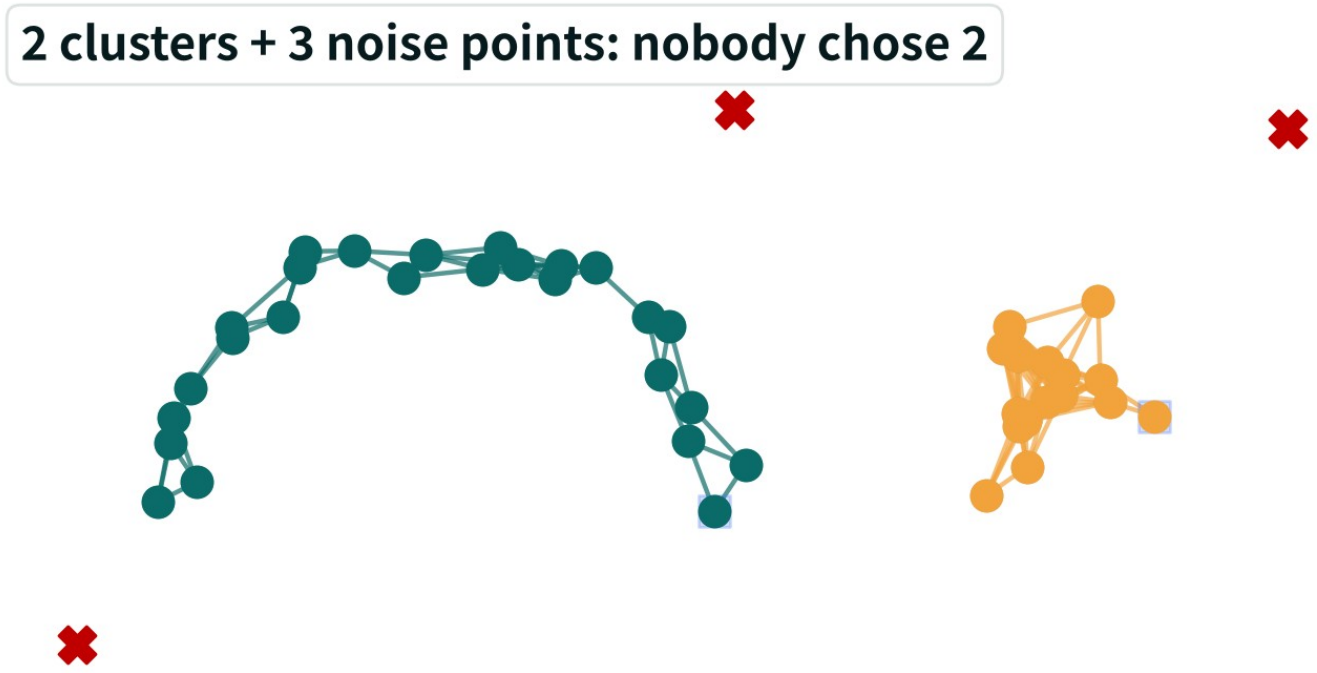
The number of clusters is the number of crowds found

When one cluster stops growing, the next starts at an unused core point

Step 7 · Count

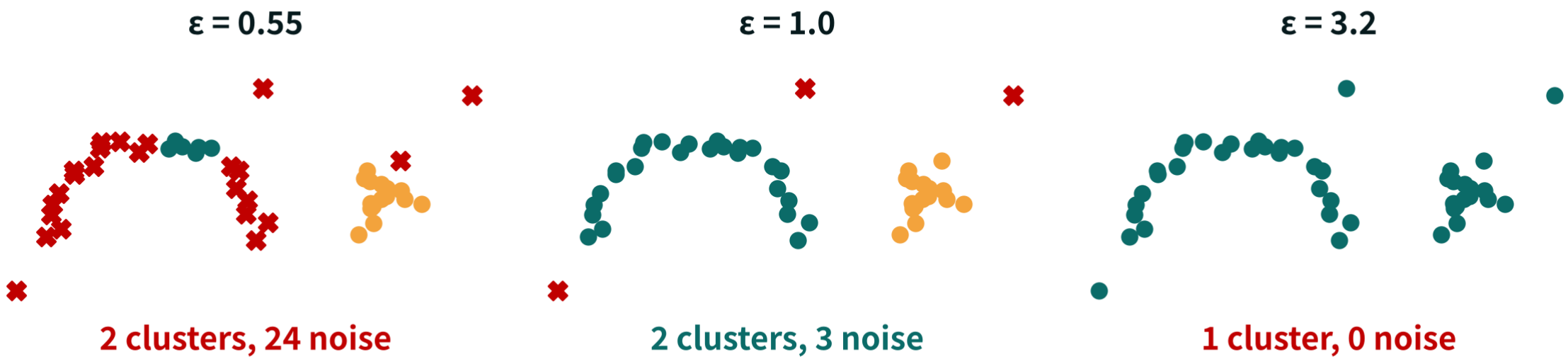
- Restart at unused core
- 2 crowds → 2 clusters
- Loners stay noise

The data chose 2



ε works like a water level: it sets how many islands appear

Same data, three values of ε, minPts = 4



※ Too small: crowds break up into noise. Too large: everything merges into one cluster.

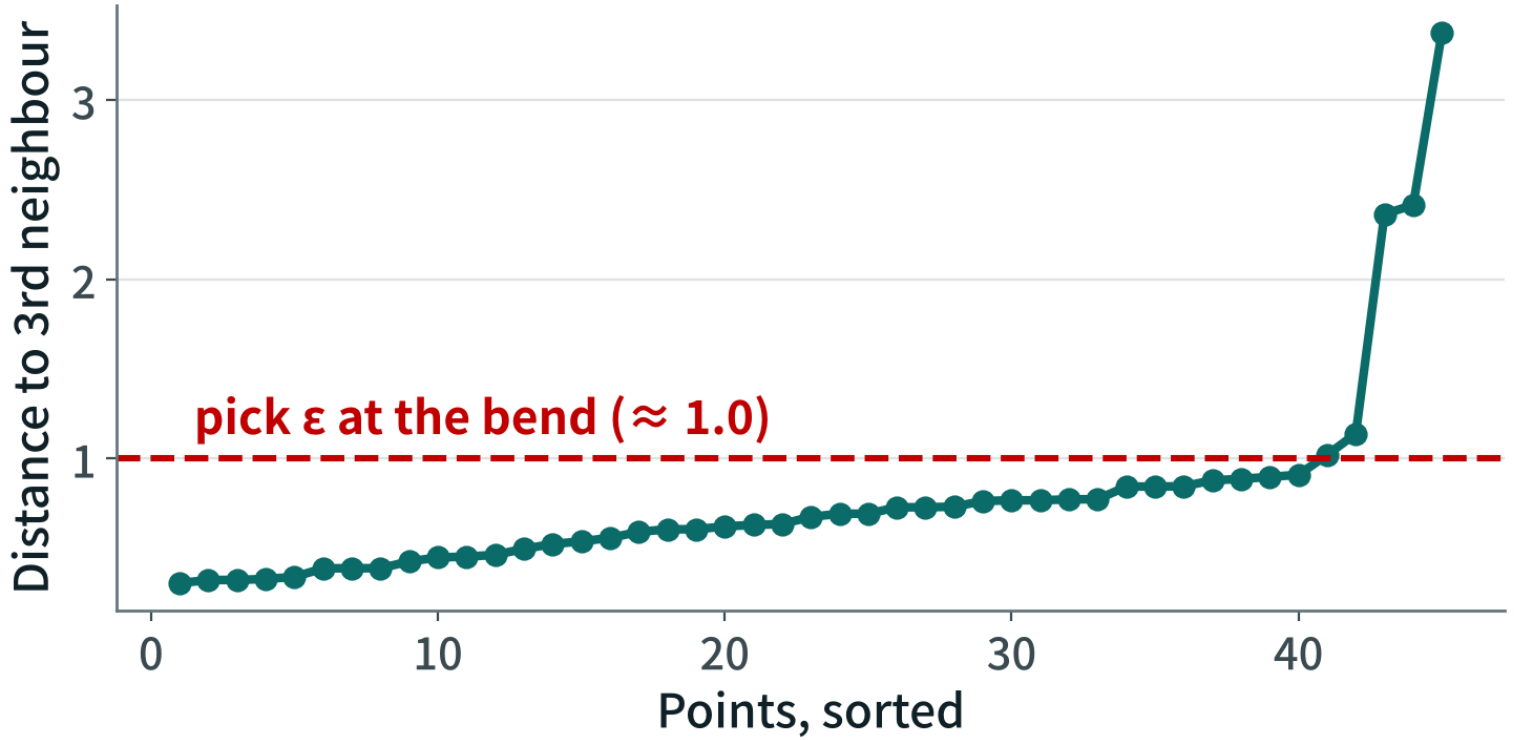
A k-distance plot suggests a good ε

For each point, the distance to its 3rd nearest neighbour, sorted from small to large

How to read it

- Flat part = crowds
- Sharp rise = loners
- Pick ε at the bend

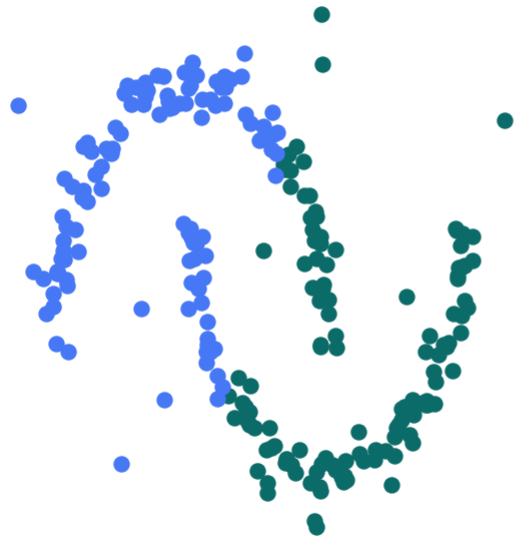
Like the K-Means elbow



Each algorithm sees a different shape of cluster

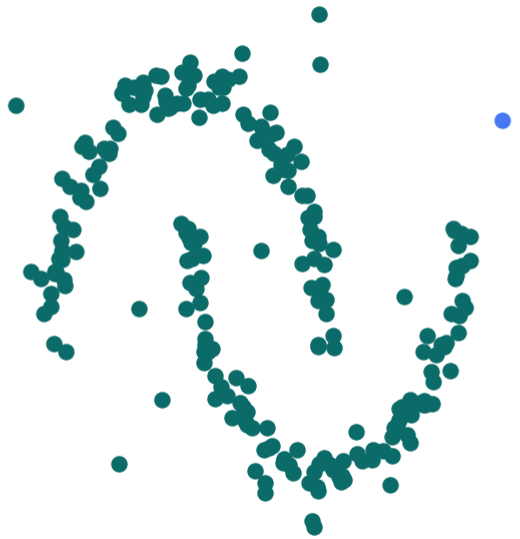
Same data, three algorithms. Colour = cluster, red X = noise

K-Means (K = 2)



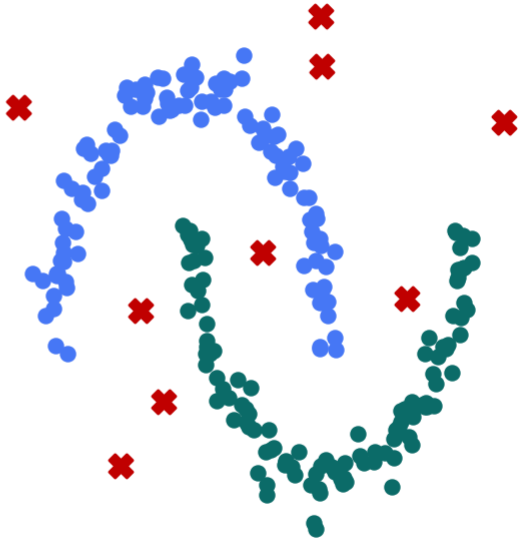
Cuts straight through the moons

Hierarchical (single linkage)



Follows the chain, but noise sticks

DBSCAN (ε = 0.2, minPts = 5)



Right shapes and noise flagged

Choose the clustering method by what you know in advance

Same goal, different assumptions about the shape and the number of groups

Method	Choose K?	Shape it finds	Outliers	Finance use
K-Means	Yes	Round blobs	Forced in	Regimes, segments
Hierarchical	Cut later	Tree of groups	Forced in	Stock trees, HRP
DBSCAN	No (ε, minPts)	Any dense shape	Flagged	Odd days, fraud

No method is right in general.
Check that the groups make economic sense.

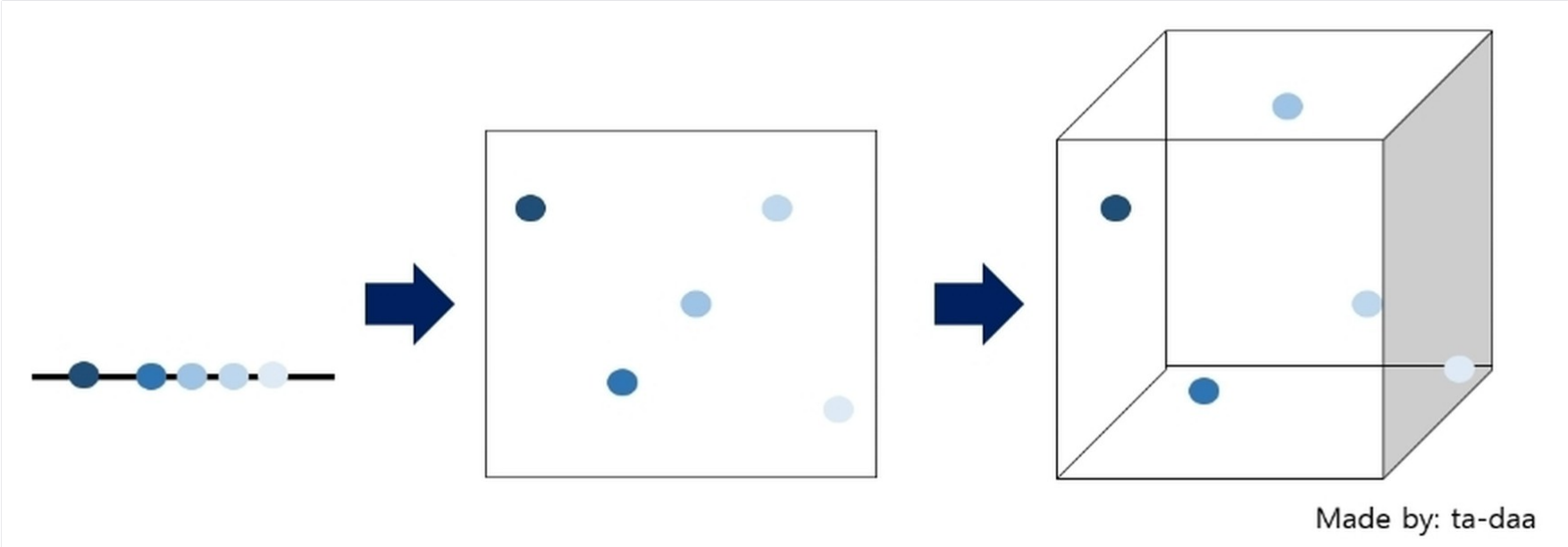
02

Shrinking Features

Curse of dimensionality · PCA · t-SNE

The same five points thin out as each dimension is added

Curse of dimensionality: line → square → cube, with the data unchanged



1-D: crowded **2-D: gaps** **3-D: empty space**

Every new feature multiplies the empty space, so distances lose meaning. Clustering and KNN suffer; keep only the informative axes.

PCA finds the directions in which the data spreads the most

Principal Component Analysis. Everyday picture: turning a torch until the shadow is longest

WHAT IT DOES

Builds new axes

Ranked by spread

EVERYDAY PICTURE

Longest shadow

Best light angle

YOU CHOOSE

How many to keep

Often 80–90% spread

Many correlated features become a few strong ones.

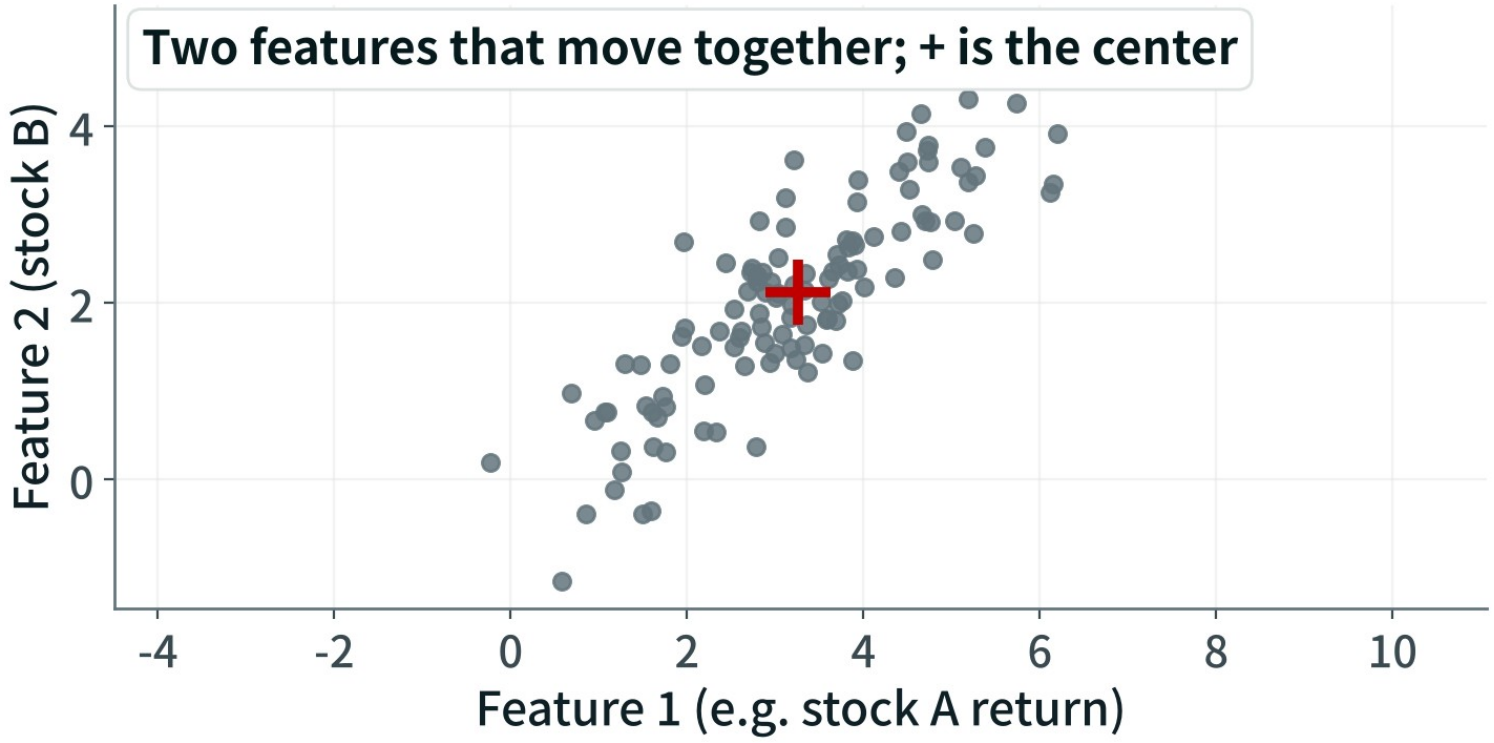
We start with two features that move together

120 days of two related stock returns (simulated)

Step 1 · The data

- Each dot is one day
- Up-right = together
- + marks the average

2 numbers per day



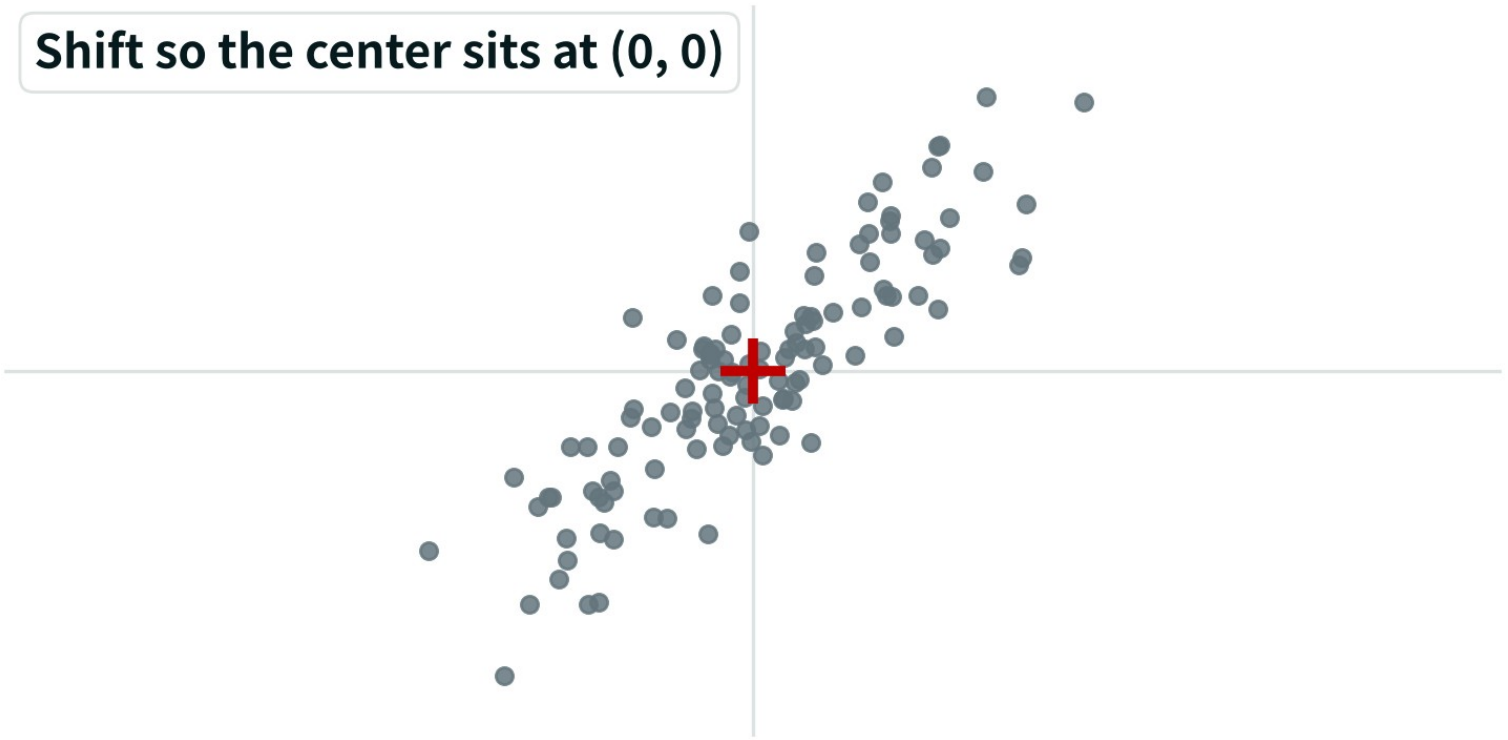
First the cloud is shifted so its center sits at zero

Subtract each feature's average; the shape stays the same

Step 2 · Center

- Subtract the average
- Shape is unchanged
- Directions start at 0

Called centering



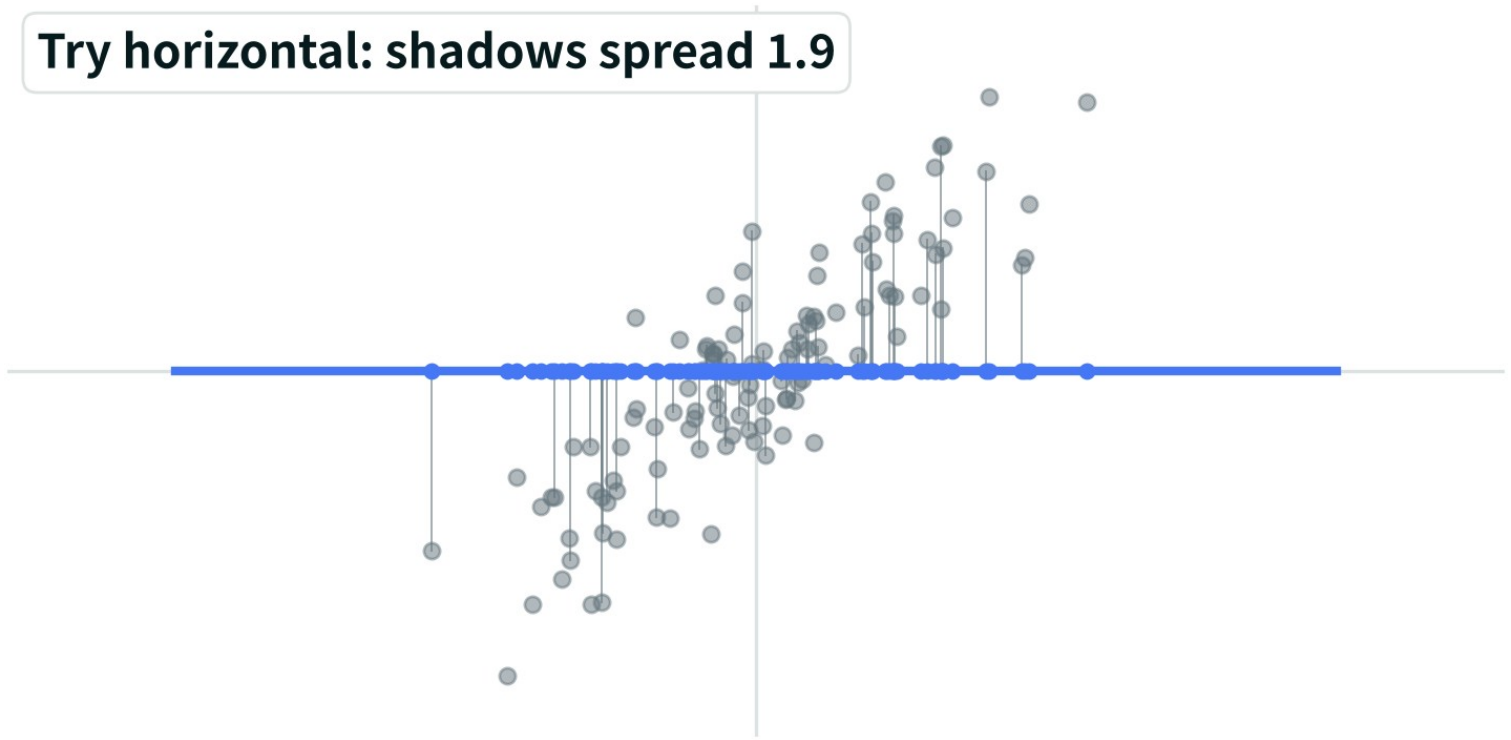
Each point casts a shadow on a line, and the shadows have a spread

Drop every dot straight onto the line; blue dots are the shadows

Step 3 · Try a line

- Shadow = projection
- Spread here = 1.9
- Wider = more kept

Spread = variance



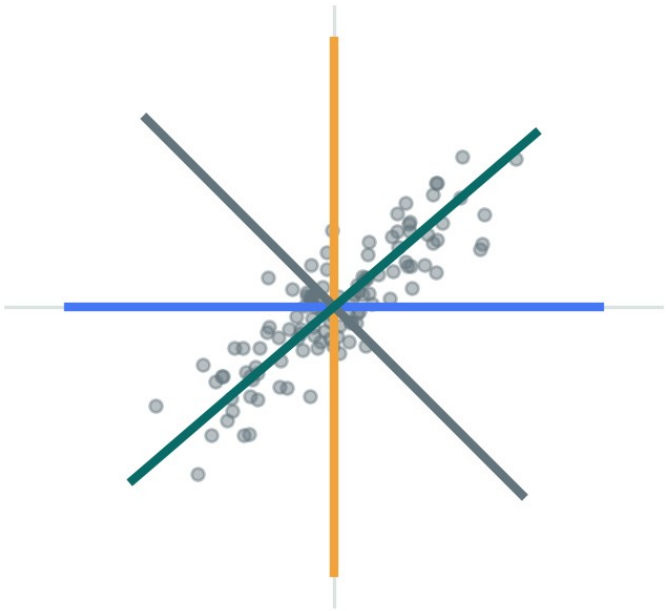
Trying every direction reveals the widest one

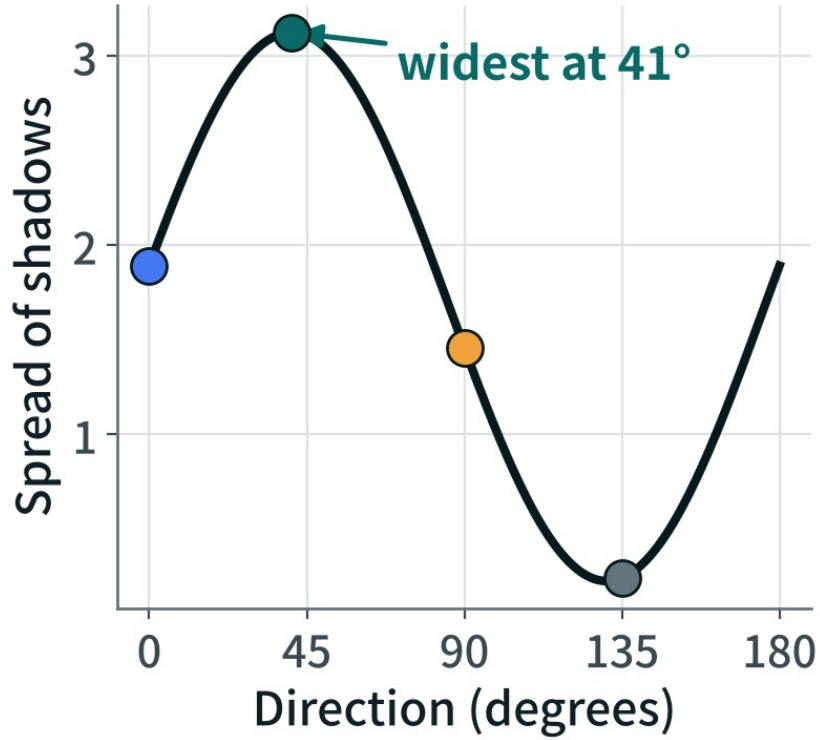
Left: four of the lines tried. Right: shadow spread for every angle from 0° to 180°

Step 4 · Search

- Turn the line 0° → 180°
- Measure each spread
- Widest near 41°

That line = PC1





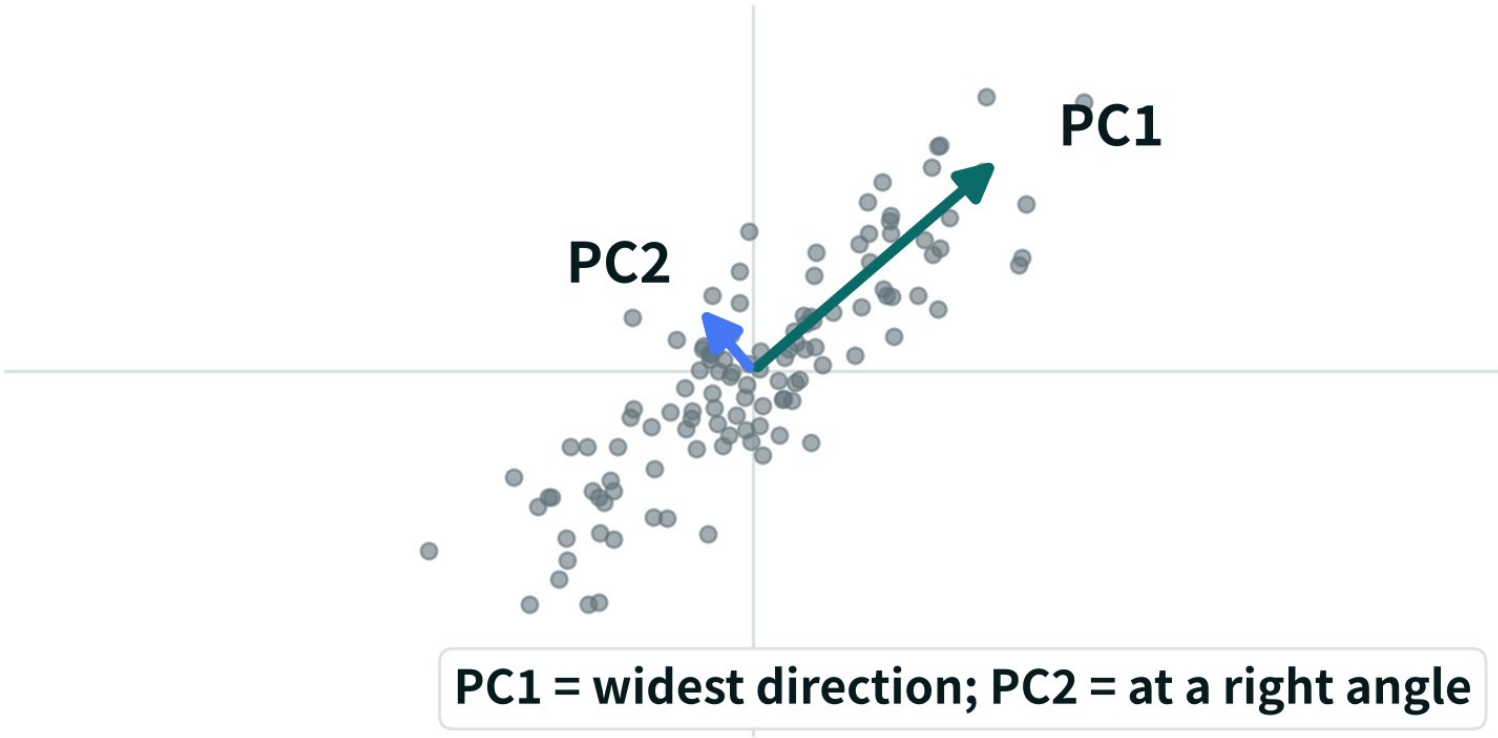
PC1 runs along the widest spread and PC2 runs across it

PC = principal component; arrow length reflects how much the data spreads that way

Step 5 · New axes

- PC1: most spread
- PC2: at 90°, the rest
- Axes mix features

Data on new axes



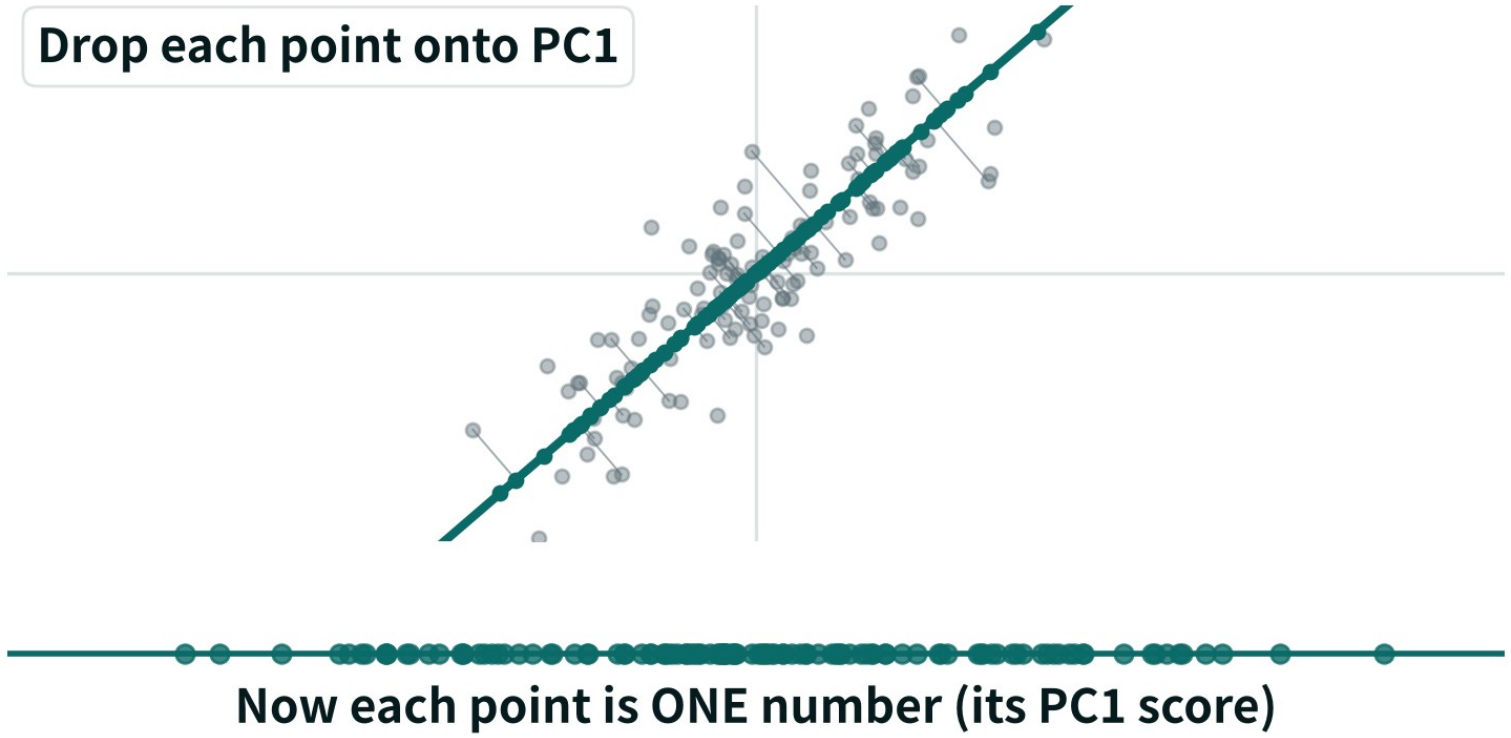
Keeping only PC1 turns two numbers into one

Each point is replaced by the position of its shadow on PC1

Step 6 · Project

- Dot → its shadow
- Position = PC1 score
- 2 columns → 1

Dimension reduced



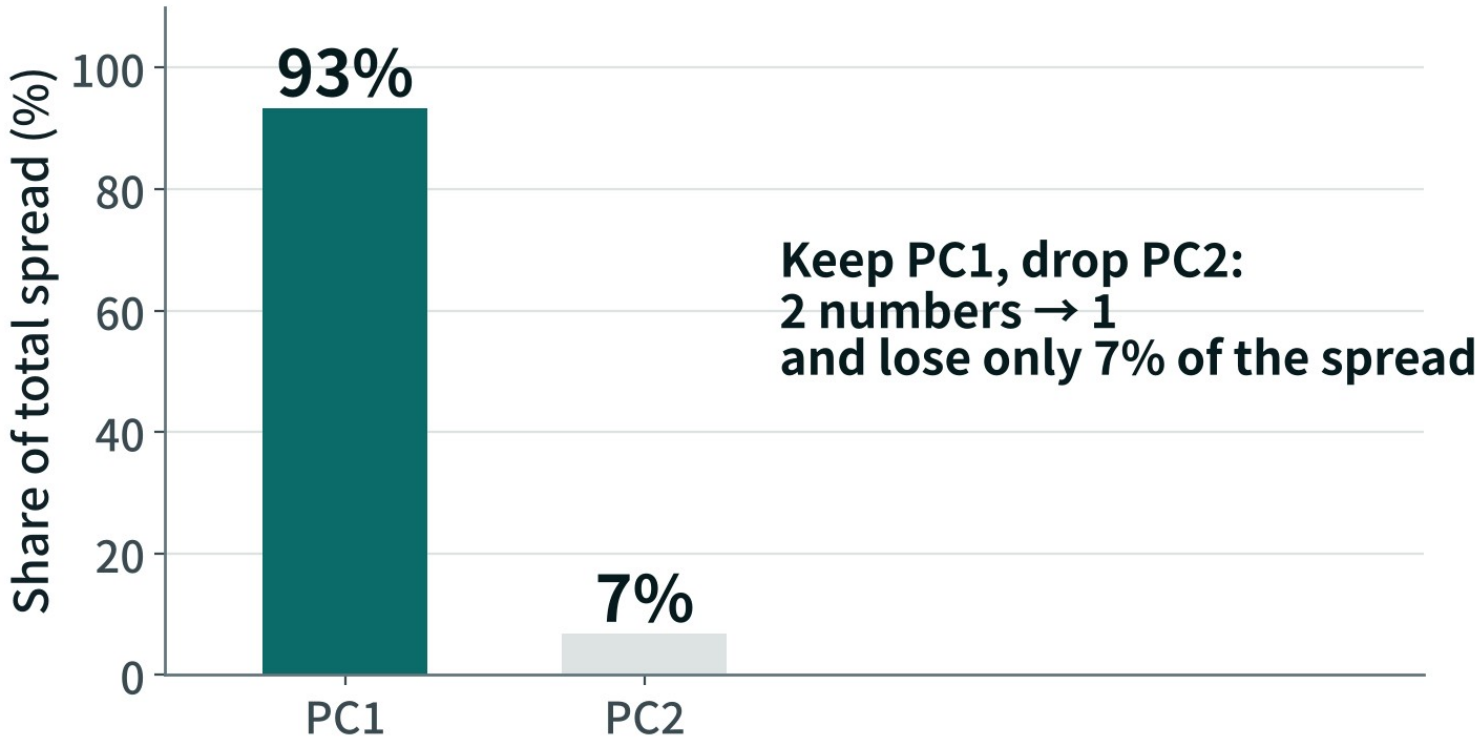
Here PC1 alone keeps 93% of the spread

Share of each component = its spread divided by the total spread

Step 7 · How much kept

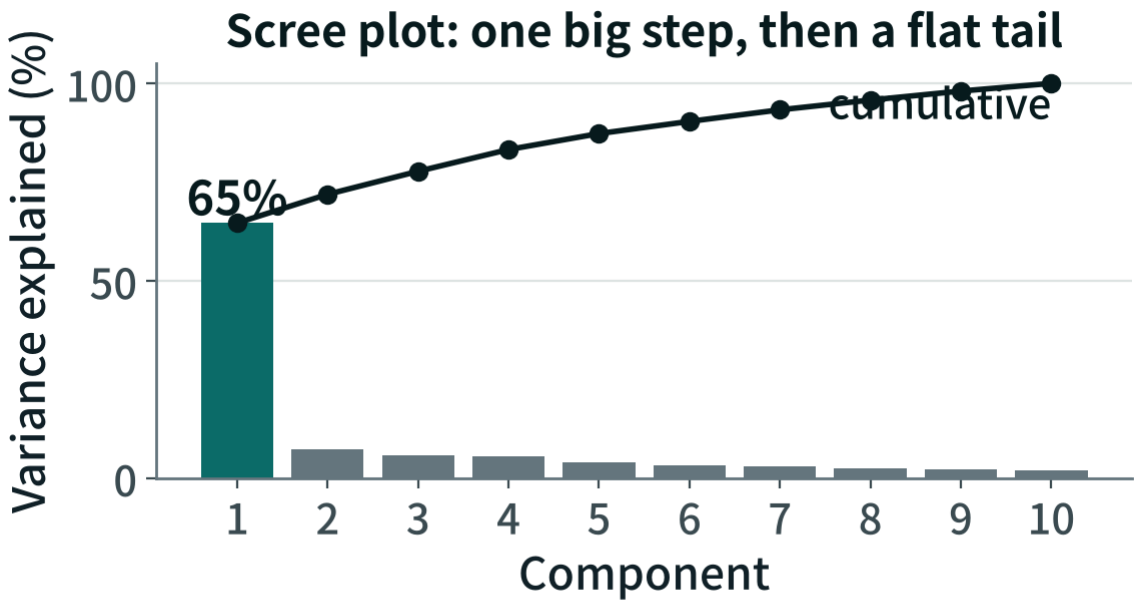
- PC1 93%, PC2 7%
- Drop PC2, lose little
- Needs correlation

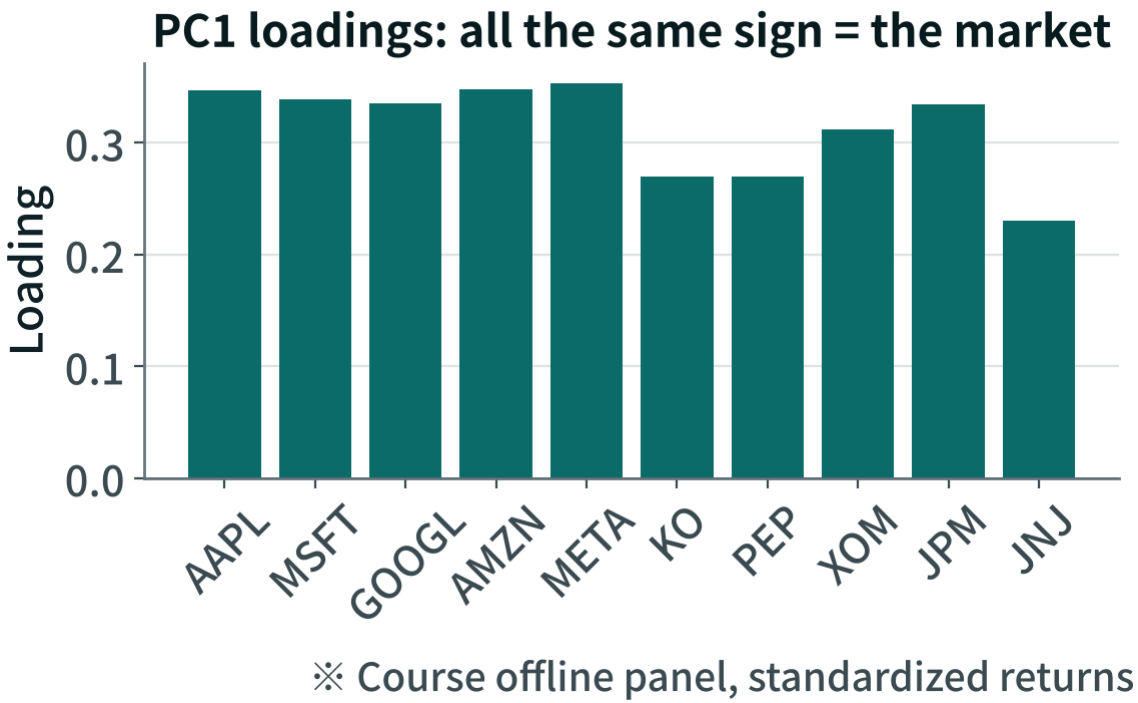
Compression worked



On stock returns the first component is the market

PCA on ten standardized daily return series





※ Course offline panel, standardized returns

※ PC1 explains about 65% here. Later components are sector tilts, then noise.

t-SNE draws a flat map that keeps neighbours next to each other

t-distributed Stochastic Neighbor Embedding. Everyday picture: seating a class so friends sit together

WHAT IT DOES

Makes a 2-D map

For looking only

EVERYDAY PICTURE

A seating plan

Friends sit close

YOU CHOOSE

Perplexity

≈ friends per point

It keeps who is near whom, not exact distances.

We start from points that live in more dimensions than we can draw

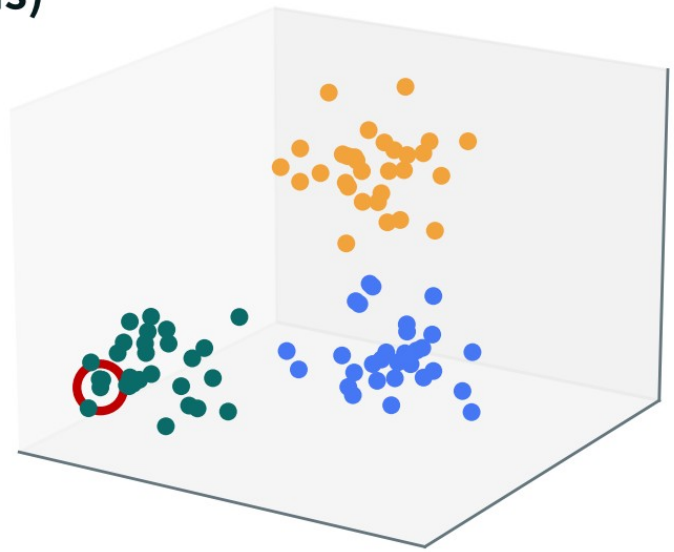
90 points in 3-D here; real data often has dozens of dimensions

Step 1 · The data

- 3 groups hidden in 3-D
- Colours only to check
- Goal: a flat map

Keep neighbours close

Original data: 90 points in 3-D (colours only for us)



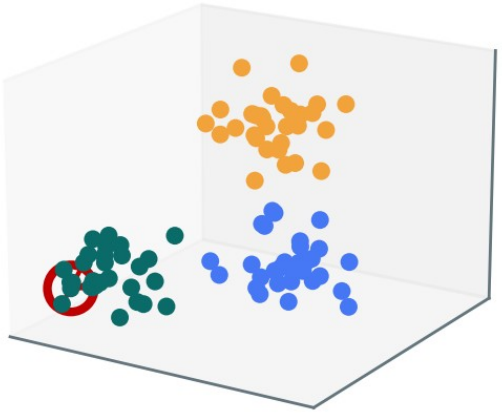
Each point lists how likely it is to pick each neighbour as a friend

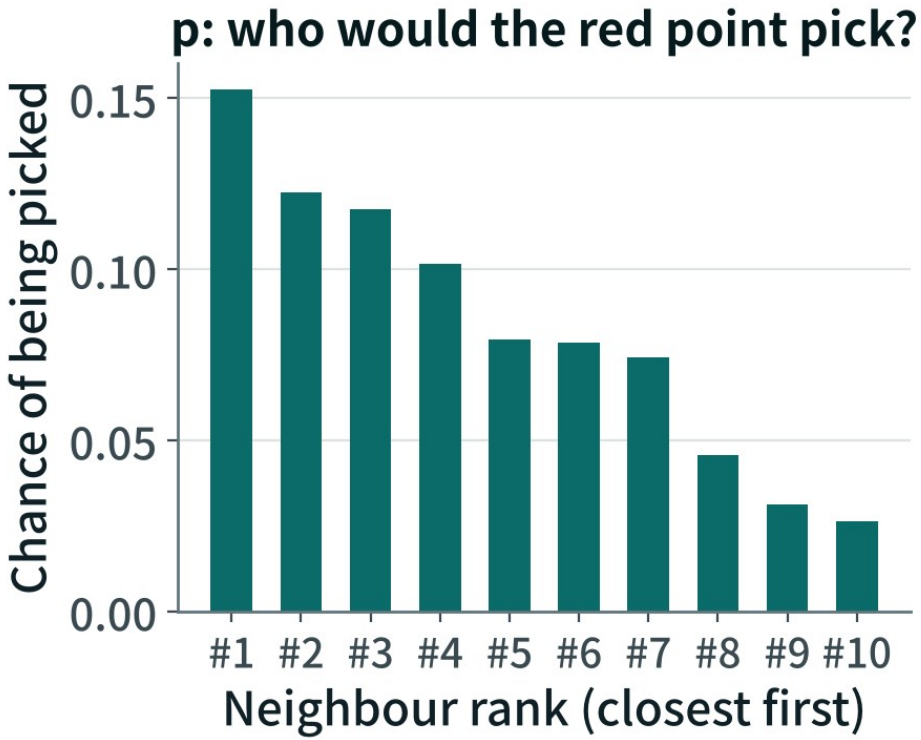
p = friend-picking chance in the original space; closer points get higher chances

Step 2 · Table p

- Red point picks a friend
- Closer → higher chance
- These chances = p

p stays fixed





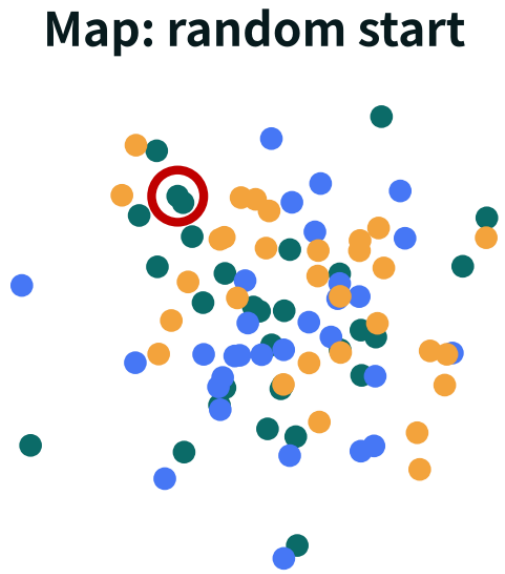
Points are scattered on the map at random, and the chances are compared

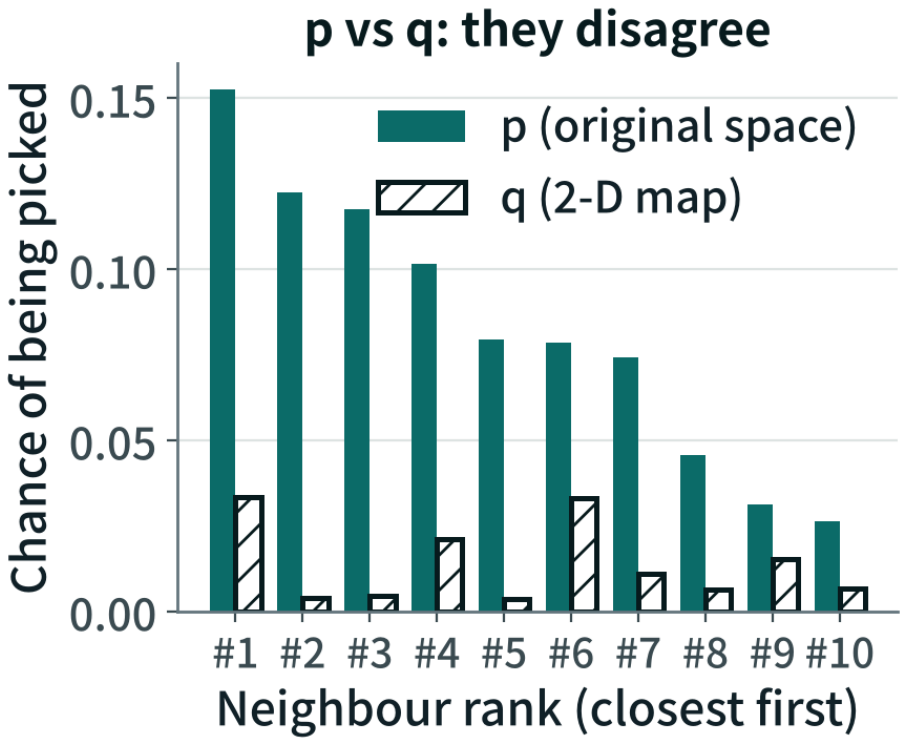
q = the same friend-picking chance, measured on the 2-D map

Step 3 · Table q

- Random map: wrong q
- Solid = p, hatched = q
- Mismatch = KL

q must copy p





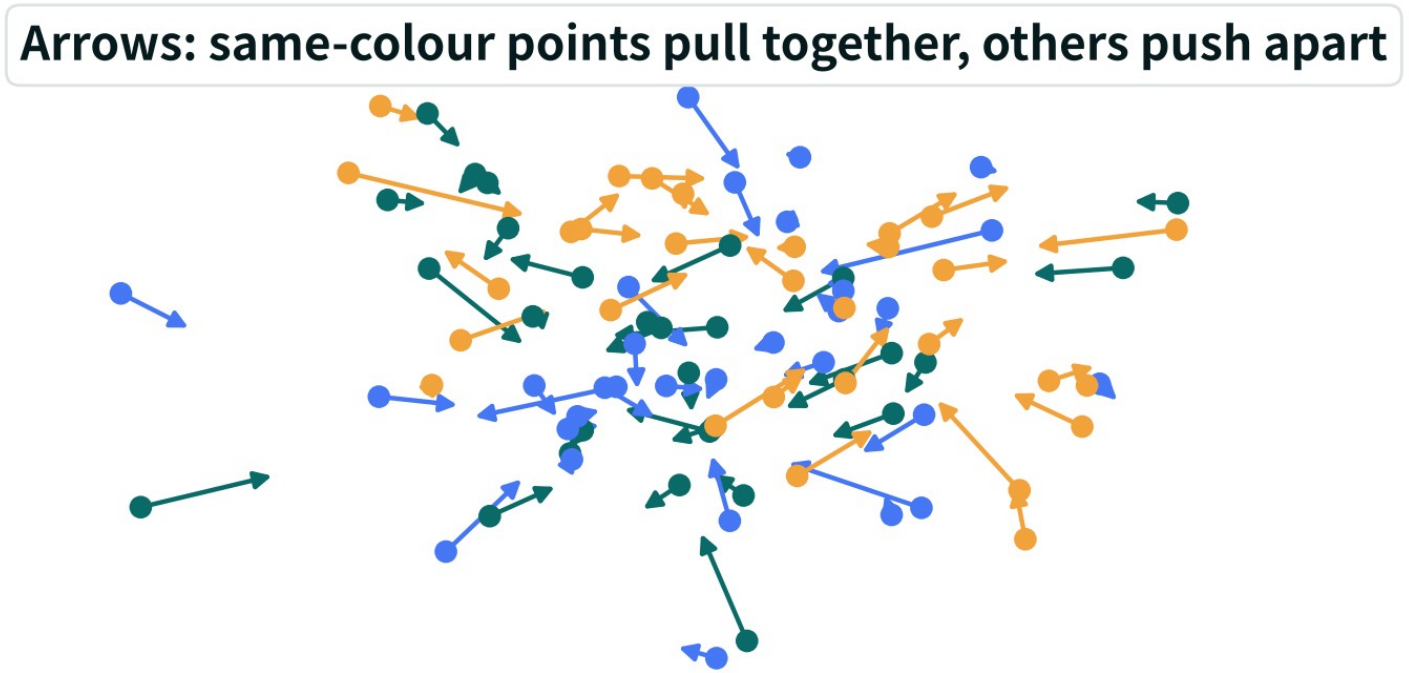
True neighbours are pulled together and strangers are pushed apart

Each arrow shows where a point moves next to shrink the mismatch

Step 4 · Pull and push

- p > q: pull closer
- p < q: push away
- Arrows = next move

Slope of KL decides



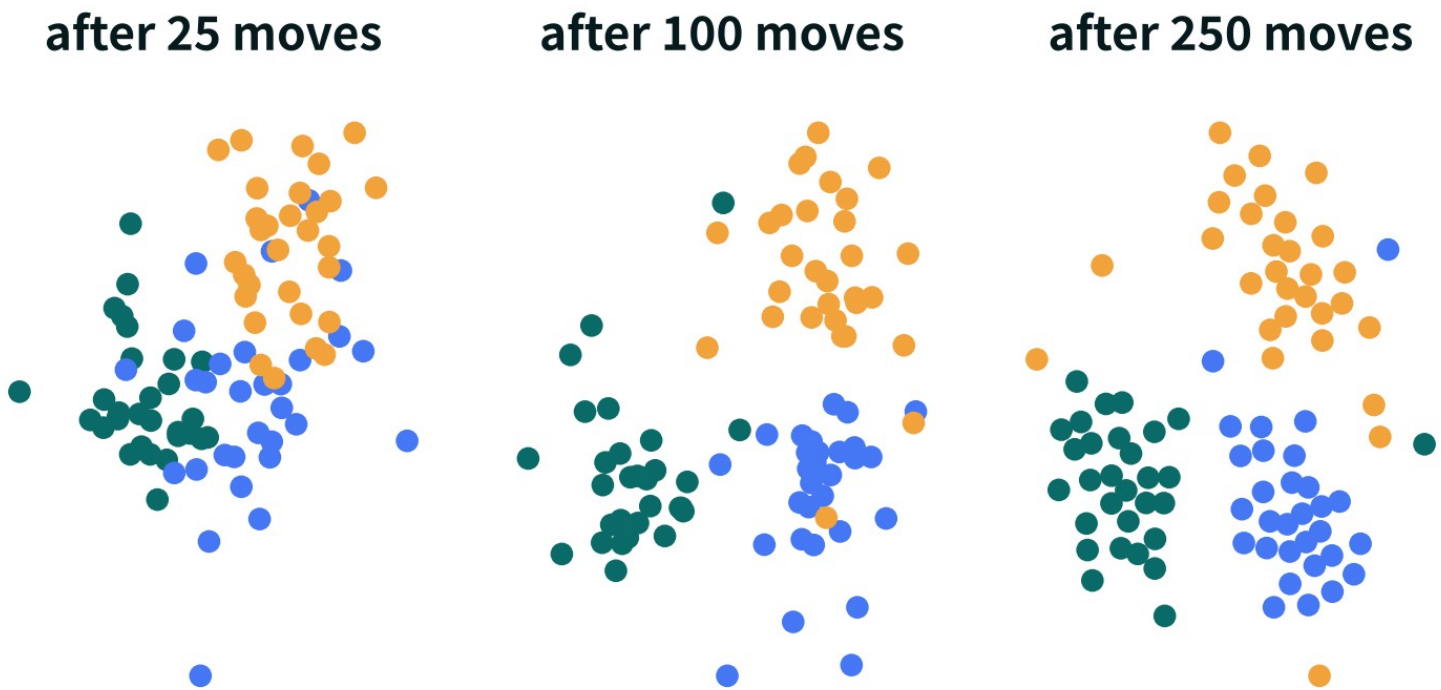
Repeating small moves lets the groups appear

The same map after 25, 100 and 250 moves

Step 5 · Repeat

- Many small moves
- Early moves pull harder
- Islands start to form

Shape emerges



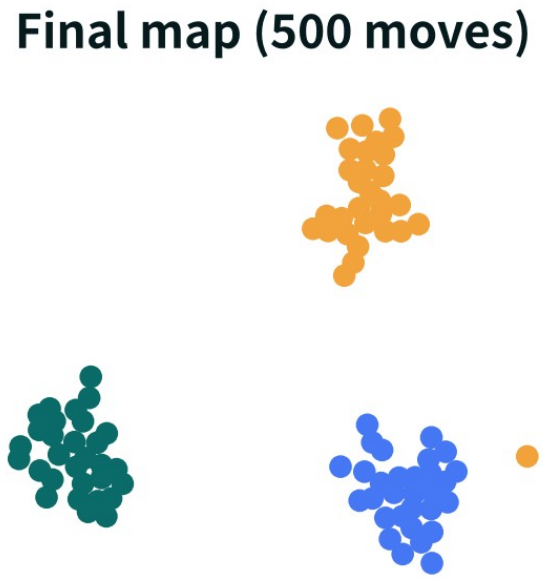
The mismatch keeps shrinking until the map settles

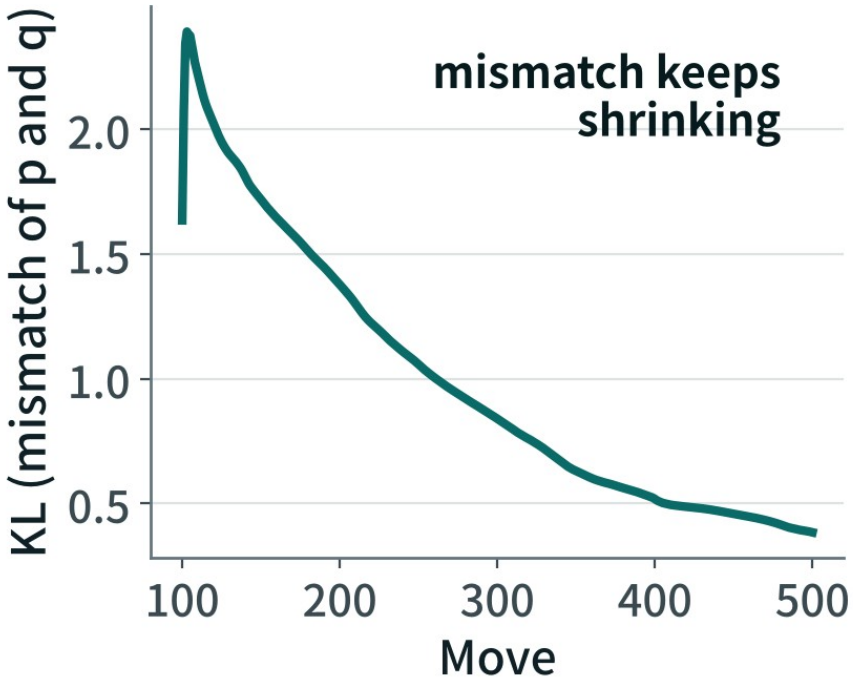
Right: KL between p and q after the warm-up moves

Step 6 · Settle

- Three clean islands
- KL keeps falling
- Stop after ~500 moves

Map finished





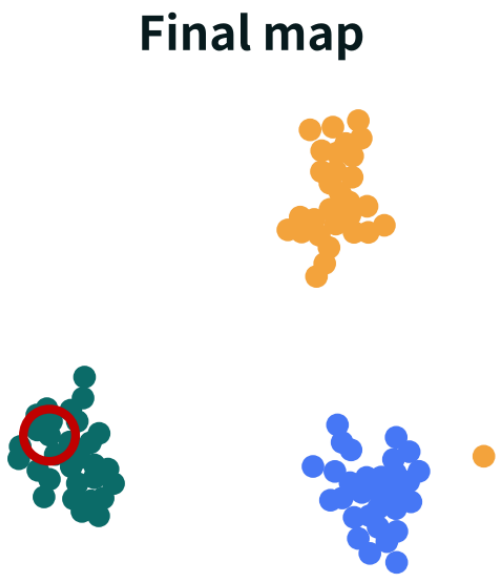
On the finished map the friend-picking chances nearly match

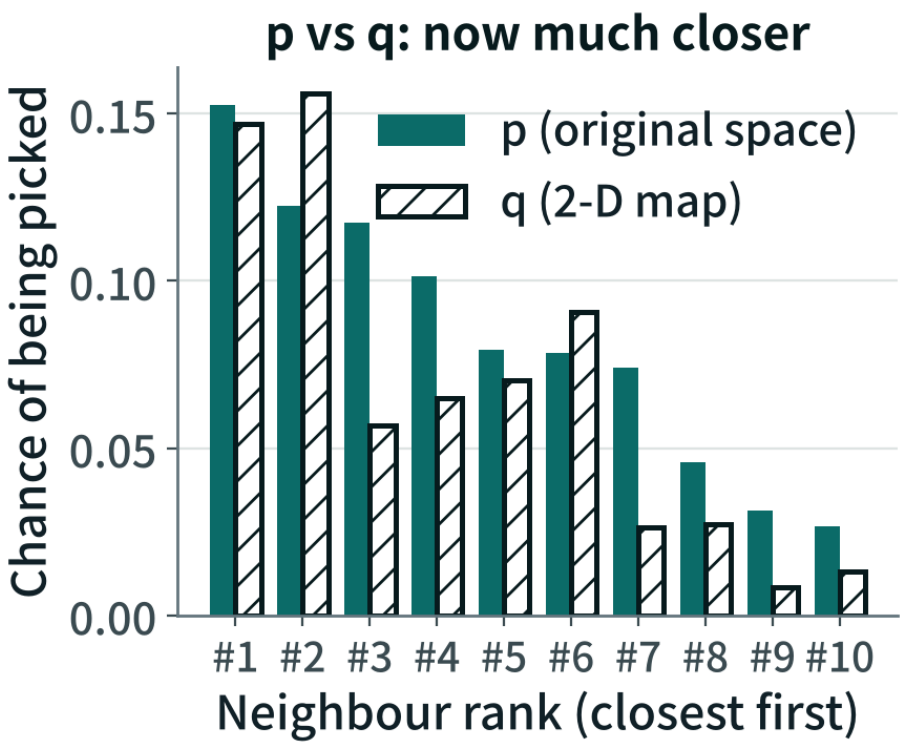
Same red point as in step 2

Step 7 · Check

- Hatched ≈ solid bars
- Neighbours kept
- Distances not kept

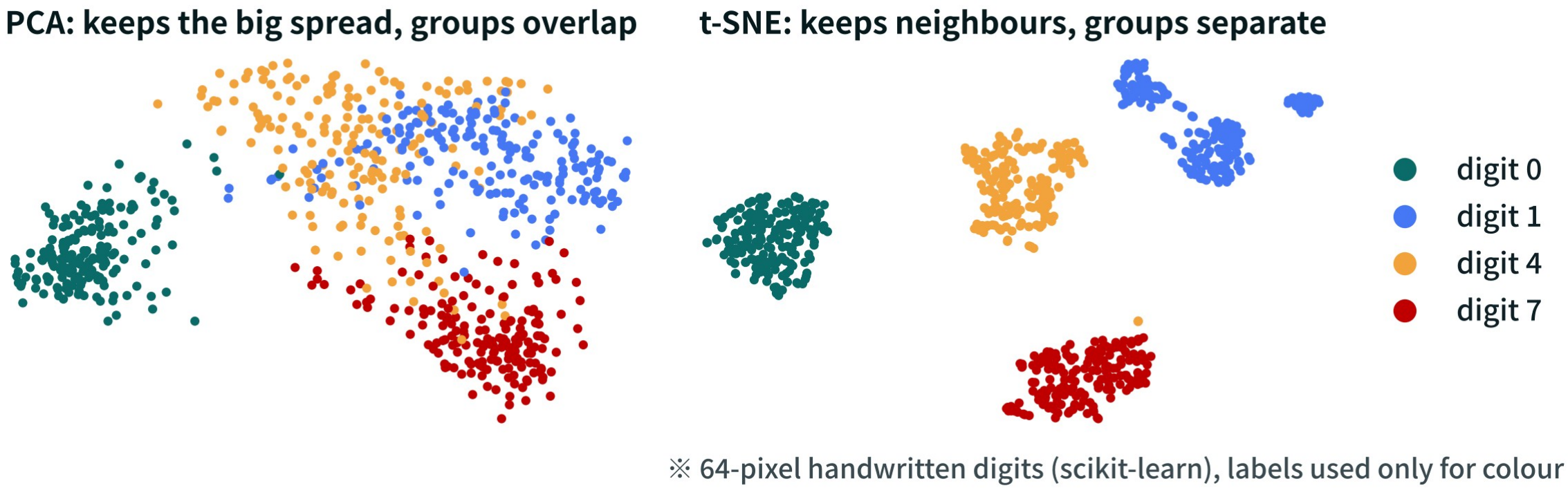
Read islands, not gaps





PCA keeps global spread while t-SNE keeps local neighbours

Same 64-dimensional digits squeezed to two dimensions



※ 64-pixel handwritten digits (scikit-learn), labels used only for colour

※ Use PCA to compress inputs for a model; use t-SNE to look at structure.

PCA casts a shadow while t-SNE rearranges the seats

Why the digits overlap under PCA but separate under t-SNE

PCA

A shadow on a wall

Groups can overlap

t-SNE

A new seating plan

Friends stay close

THE PRICE

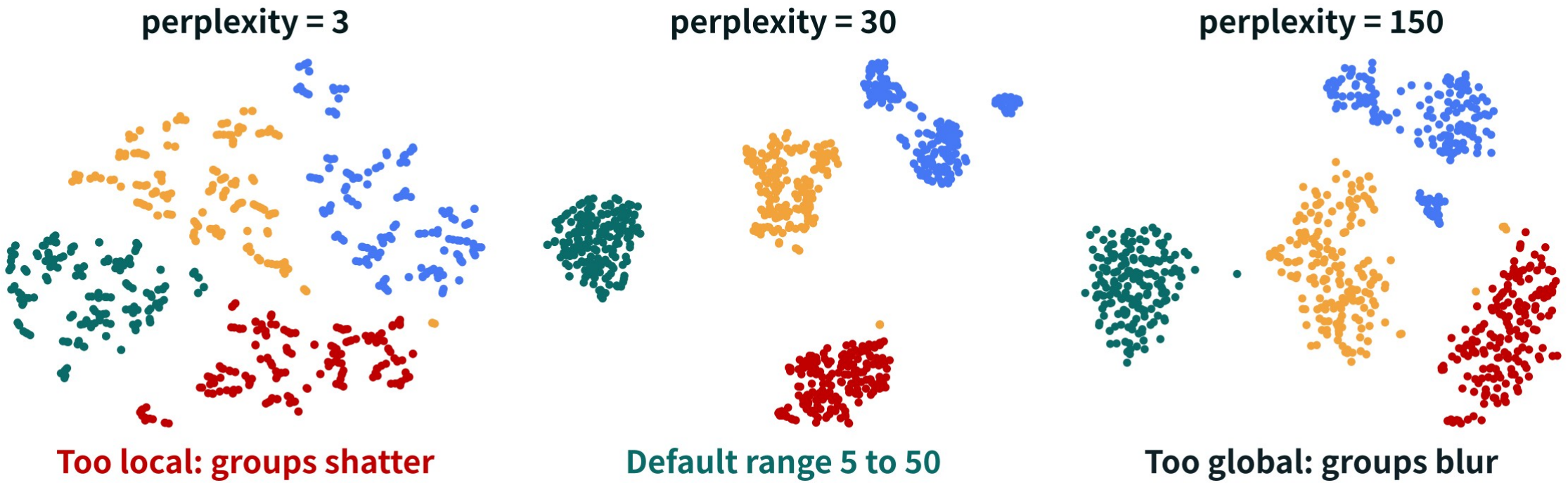
Distances bend

Map gaps mean little

PCA keeps 40% of the spread in two axes, so the digit groups overlap.
Use PCA to compress inputs; use t-SNE to look at structure.

Perplexity sets how many neighbours each point considers

Same data, three perplexity values



※ On a t-SNE map, cluster sizes and gaps between clusters carry no meaning. Never feed it to a model.

In practice PCA compresses first and t-SNE maps second

The standard recipe when there are many features: t-SNE is slow and sensitive to noise directions

Raw features → **Standardize** → **PCA to ~50** → **t-SNE to 2-D** → **Re-run and check**

WHY STANDARDIZE

Equal units

No feature dominates

WHY PCA FIRST

Faster, cleaner

Drops noise directions

WHY RE-RUN

Shapes can move

Change seed, perplexity

Trust islands that survive every re-run.

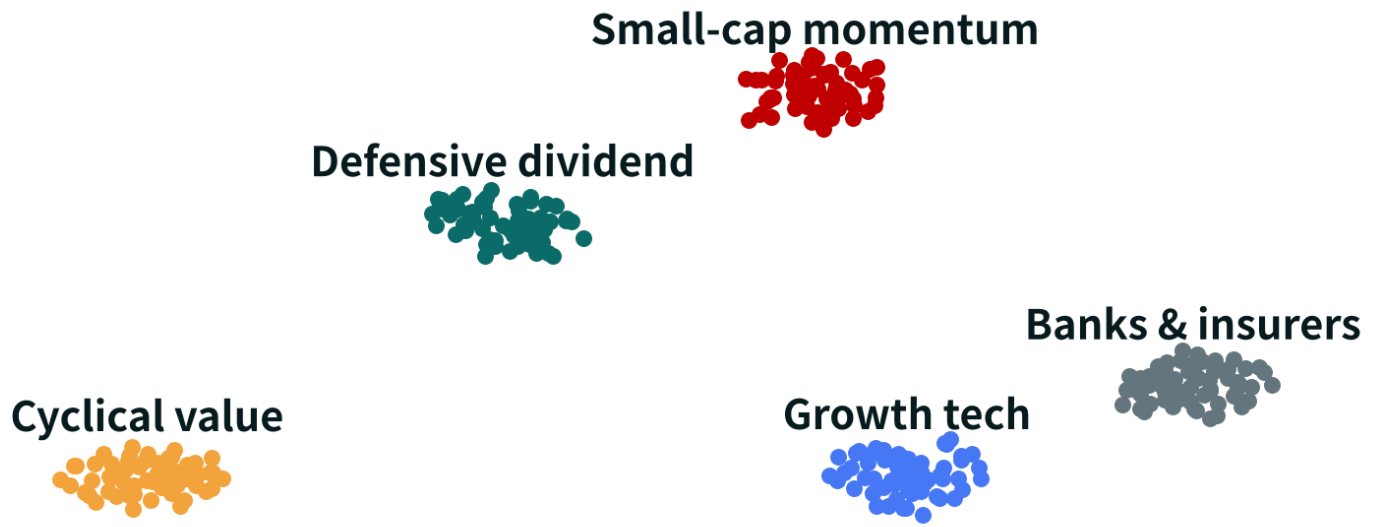
A map of stock characteristics shows style groups at a glance

Each stock described by 30 traits such as value, momentum, volatility and profitability

How to use it

- Islands = similar style
- Compare with sectors
- Confirm by clustering

Exploration, not proof



※ Simulated: 300 stocks × 30 characteristics → standardize → PCA 10 → t-SNE

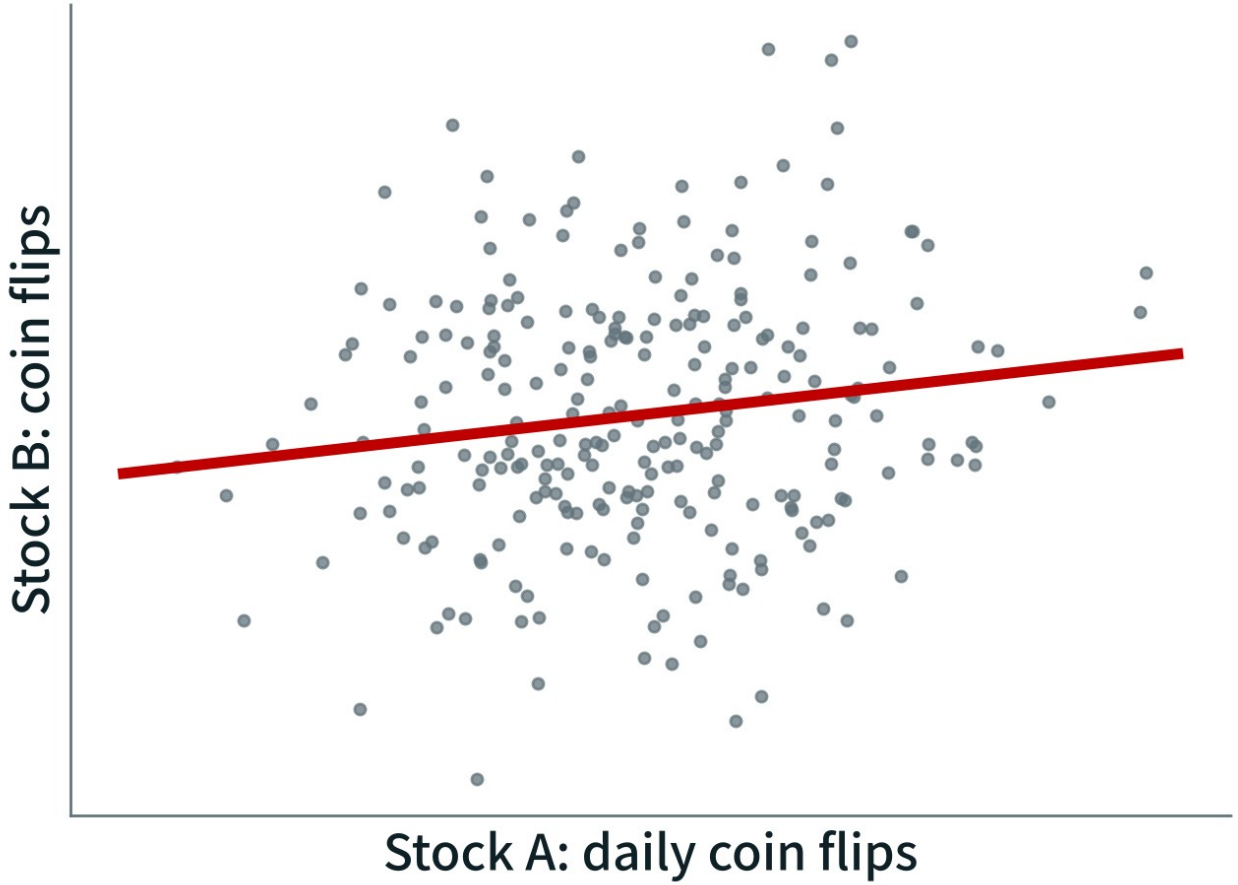
03

Cleaning Noise

Fake links, trees and grass, and a calmer portfolio

Pure chance can make two stocks look linked

Two stocks that are nothing but coin flips, watched for 250 days



Real link

0

What the data shows

0.15

→ a link that is pure luck

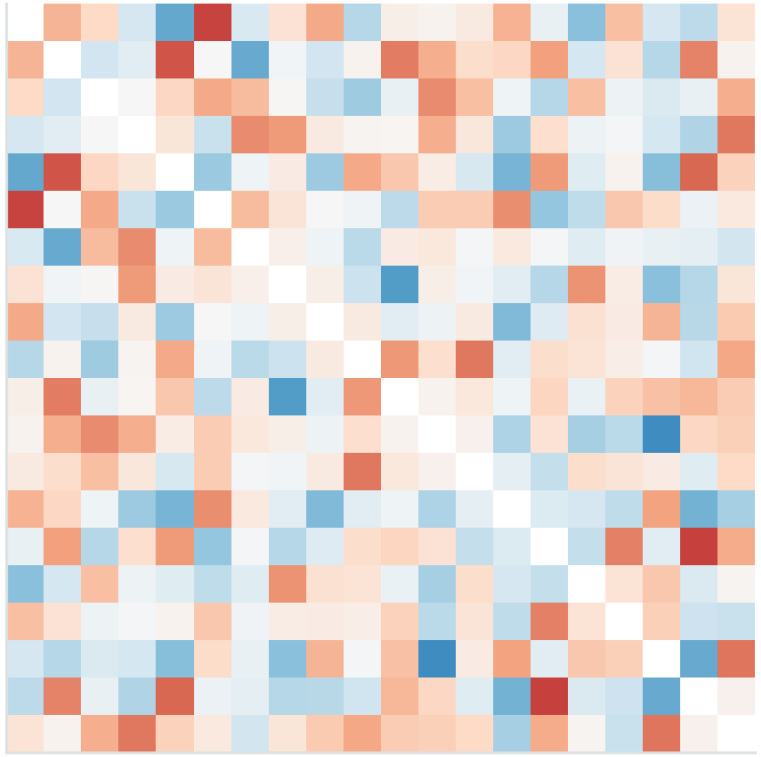
With many stocks, fake links show up everywhere

Left: what is really true. Right: what 60 days of data show

Truth: no links at all

20 stocks × 20 stocks

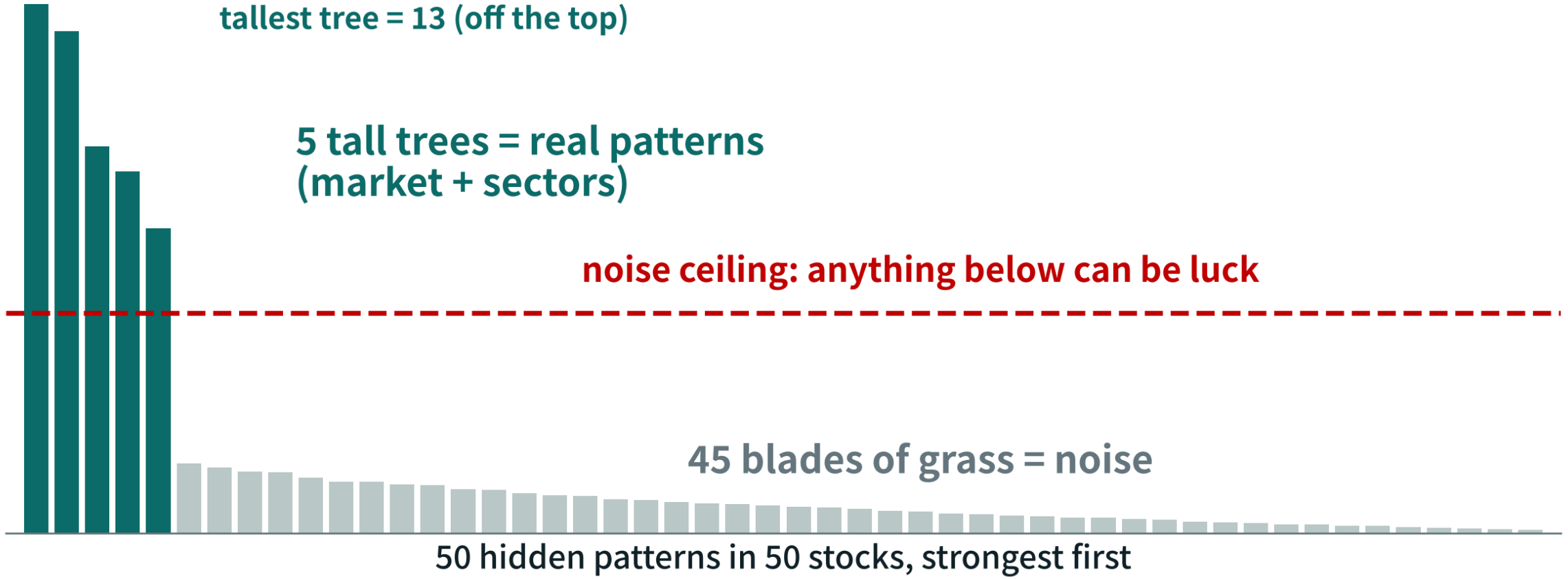
Measured: fake links everywhere



red = moved together, blue = opposite

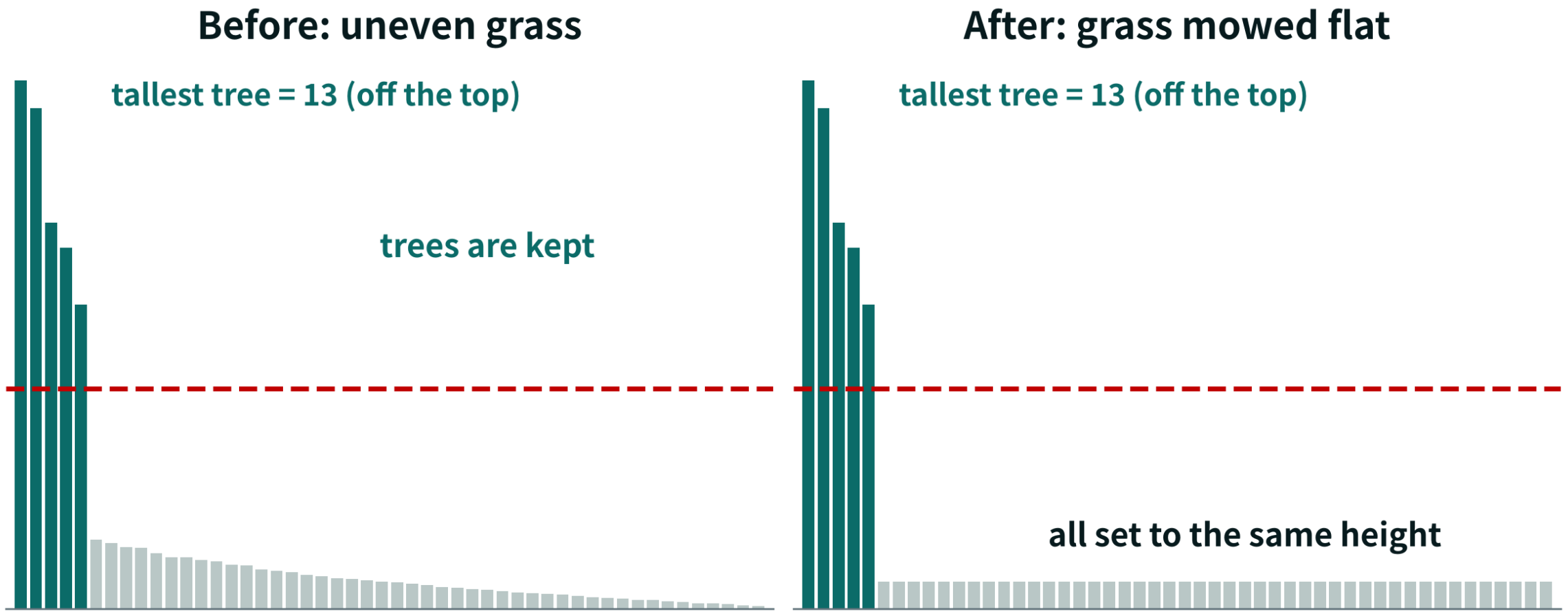
Real patterns stand tall like trees; noise is the grass

The red line is the tallest height that luck alone can reach



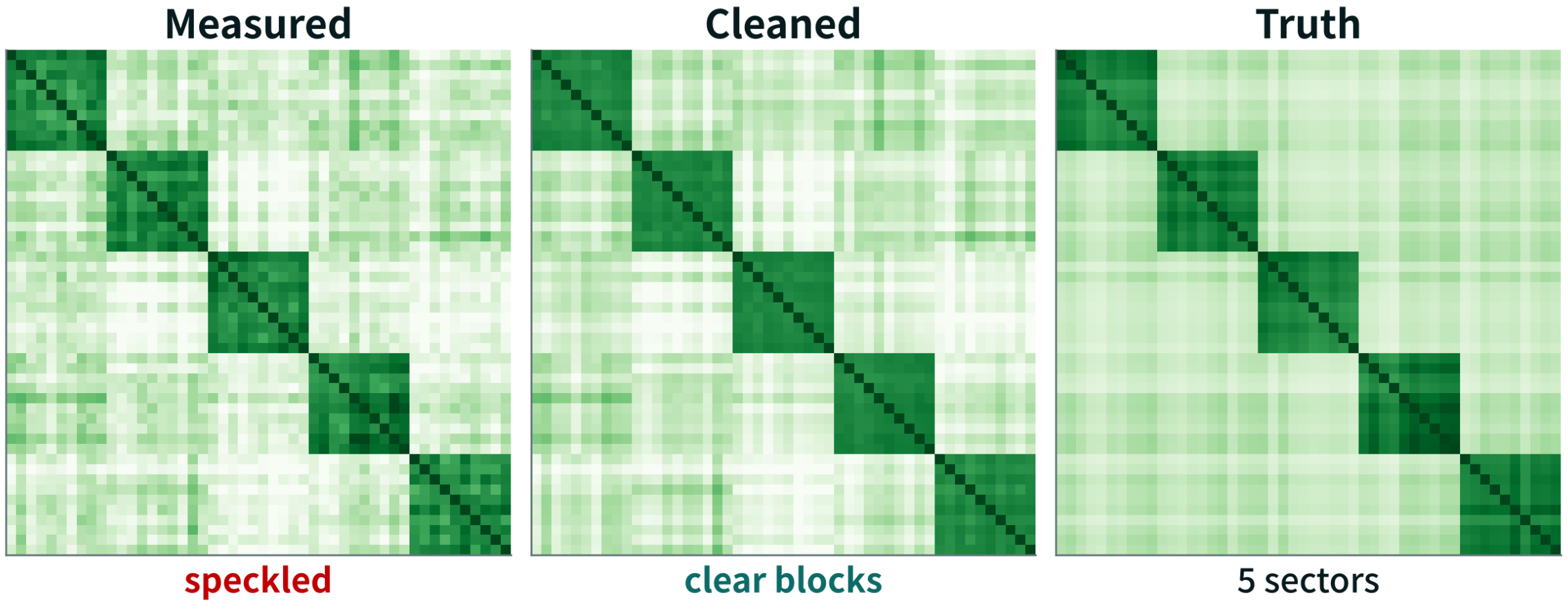
Cleaning means mowing the grass to one height

Keep the trees, flatten everything below the red line



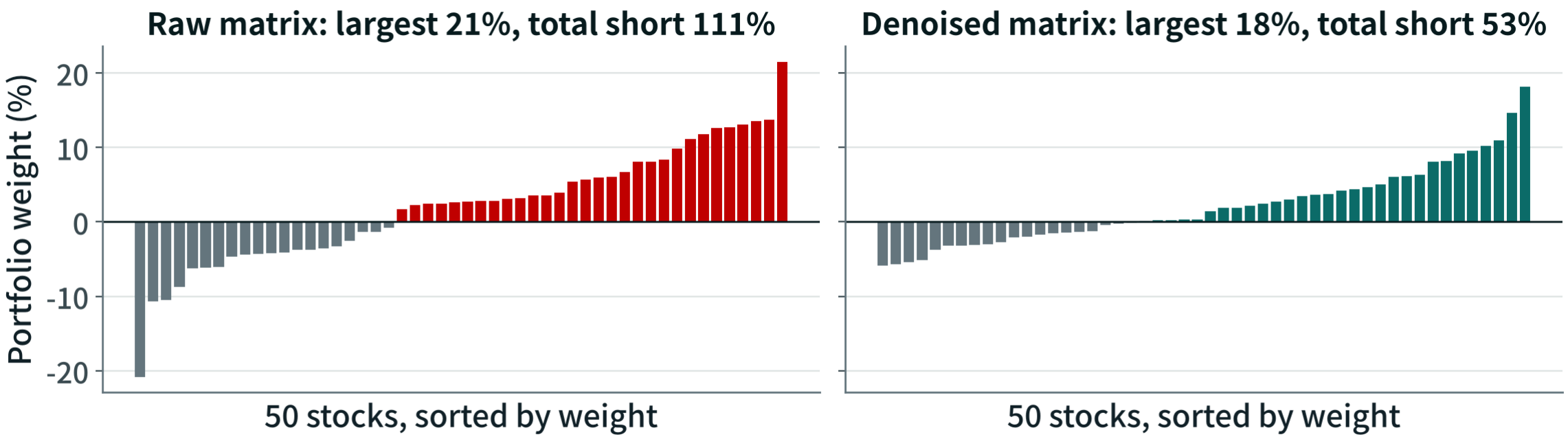
After mowing, the matrix looks like the truth

50 stocks in 5 sectors; darker green = moves together more



A clean matrix gives calmer portfolio weights

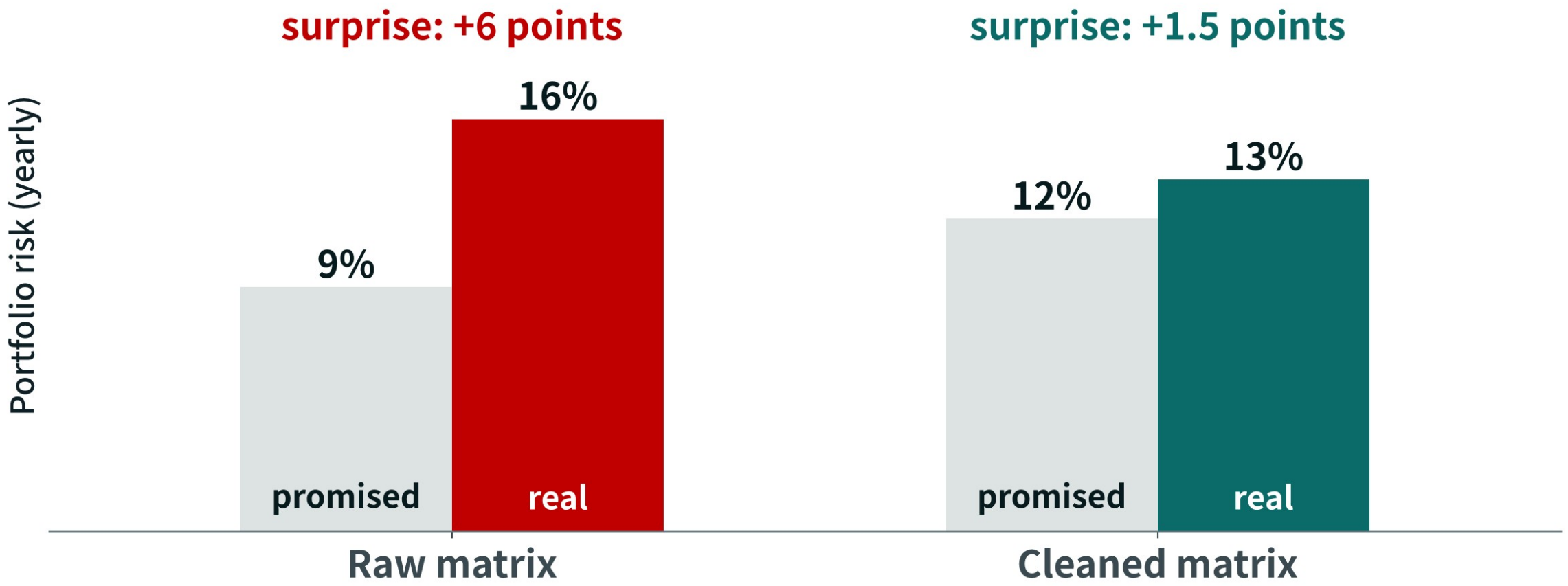
Red bars on the left are wild bets on links that are not real



※ Minimum-variance weights from one 120-day sample; grey bars are short positions

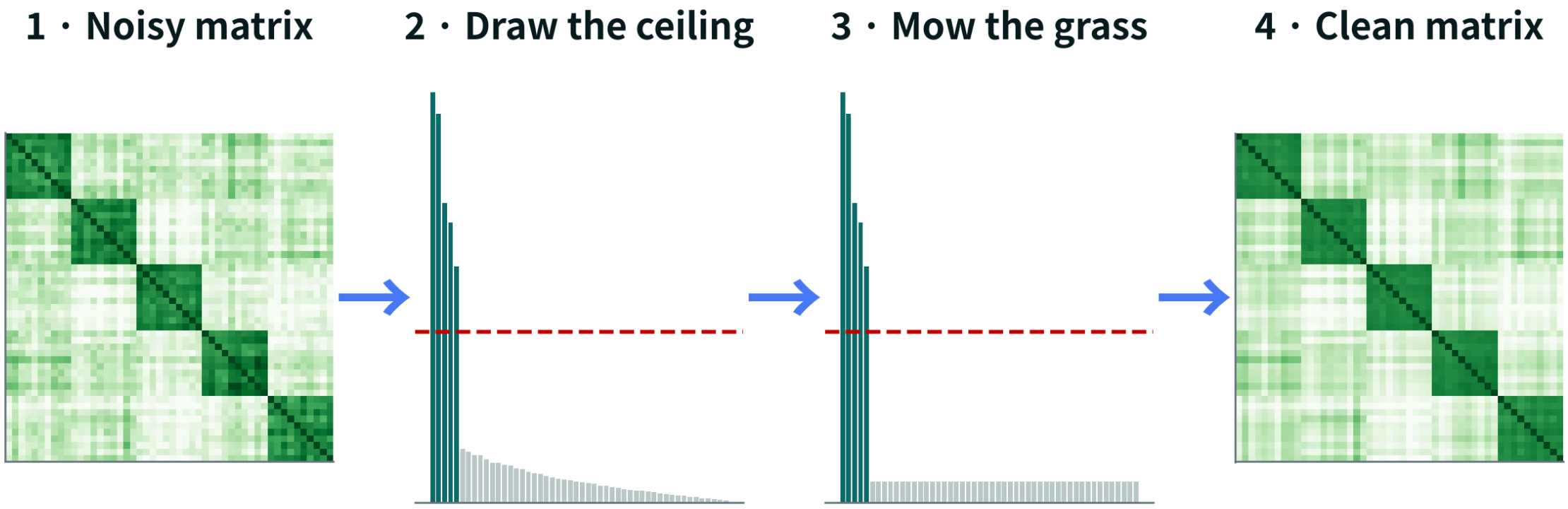
The cleaned portfolio keeps its risk promise

Promised risk vs real risk, median of 300 test runs



The whole cleaning recipe fits in one picture

Four steps from a noisy matrix to a clean one



Then feed the clean matrix to the portfolio optimizer

Six tools cover the whole unsupervised toolbox

One line each: what it does and the one knob you must set

K-MEANS

Nearest center

Knob: K (elbow)

HIERARCHICAL

Merge tree

Knob: linkage, cut

DBSCAN

Dense regions

Knob: ε, minPts

PCA

Widest axes

Knob: no. of PCs

t-SNE

Neighbour map

Knob: perplexity

NOISE CLEANING

Mow the grass

Keep only tall trees

A

Appendix: The Math

The formulas behind the pictures, for those who want them

K-Means minimises the distance to each group's center

Center (centroid) = the average point of a group; K = number of groups you choose

What it minimises

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

How a center is updated

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

STEP A

Assign

Join the nearest center

STEP B

Update

Move to the group mean

STOP

Converged

No point changes group

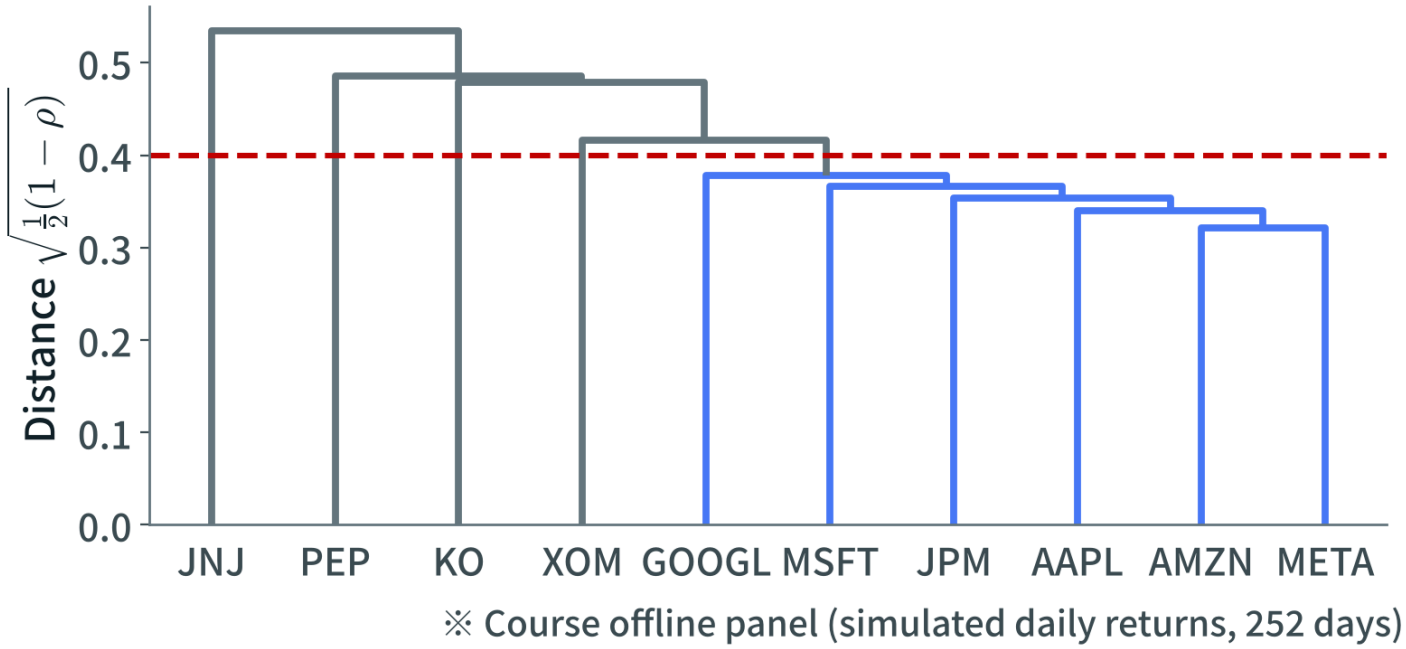
Both steps can only lower J, so the loop always stops.

Correlation distance turns a return panel into a stock family tree

Stocks that move together sit close; López de Prado (2016) uses this distance for HRP in Unit 11

Correlation distance

$$d_{ij} = \sqrt{\frac{1}{2}\left(1 - \rho_{ij}\right)}$$



ρ = 1 gives distance 0; ρ = −1 gives distance 1.
Cut the tree and sector-like groups appear.

DBSCAN defines clusters as dense regions

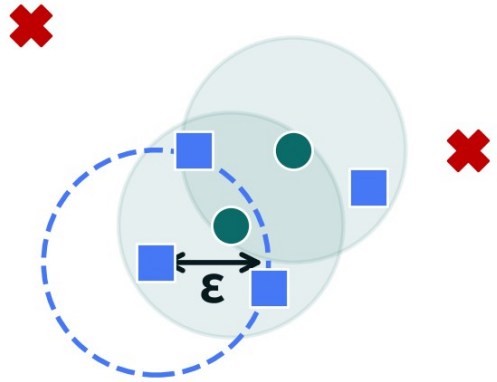
Density-Based Spatial Clustering of Applications with Noise (Ester et al., 1996)

ε-neighbourhood of point p

$$N_\varepsilon(p) = \{\, q : \|p - q\| \le \varepsilon \,\}$$

Core point rule

$$|N_\varepsilon(p)| \ge \text{minPts} \;\Rightarrow\; p \text{ is a core point}$$



- Core: ≥ 4 points within ε
- Border: near a core point
- Noise: belongs to no one

※ minPts = 4 (the point itself counts)

Two knobs, ε and minPts. No K to choose.

Correlation decides how much one component can carry

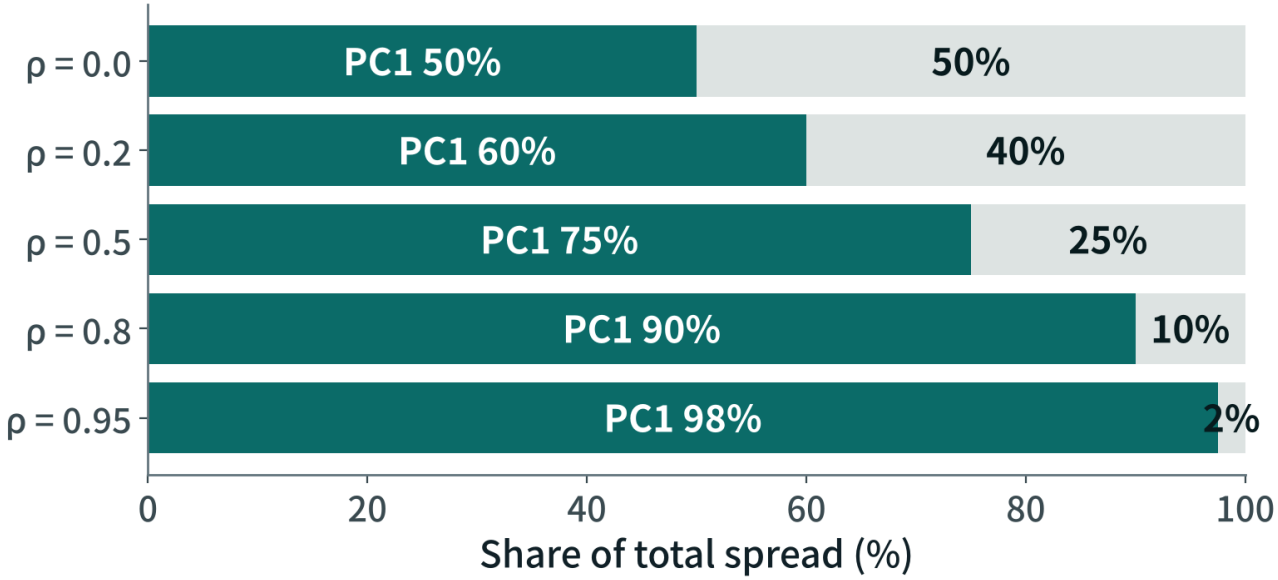
Eigenvector v = a direction; eigenvalue λ = the spread along it

PCA in one line

$$\Sigma\, v_k = \lambda_k\, v_k, \qquad \text{share}_k = \frac{\lambda_k}{\sum_j \lambda_j}$$

Two stocks with correlation ρ

$$\begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix} \Rightarrow \lambda_{1,2} = 1 \pm \rho$$



Compression pays only when features are correlated.

t-SNE matches neighbour probabilities in two spaces

van der Maaten & Hinton (2008). x = original point, y = its position on the map

$$p_{j|i} = \frac{\exp\left(-\|x_i - x_j\|^2 / 2\sigma_i^2\right)}{\sum_{k \neq i} \exp\left(-\|x_i - x_k\|^2 / 2\sigma_i^2\right)}$$

$$q_{ij} = \frac{\left(1 + \|y_i - y_j\|^2\right)^{-1}}{\sum_{k \neq l} \left(1 + \|y_k - y_l\|^2\right)^{-1}}$$

$$\mathrm{KL}(P \,\|\, Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

In plain words

① p: who is close in the data

② q: who is close on the map

③ KL: penalty if they differ

σ is set by perplexity

Perplexity is roughly the number of friends each point keeps.

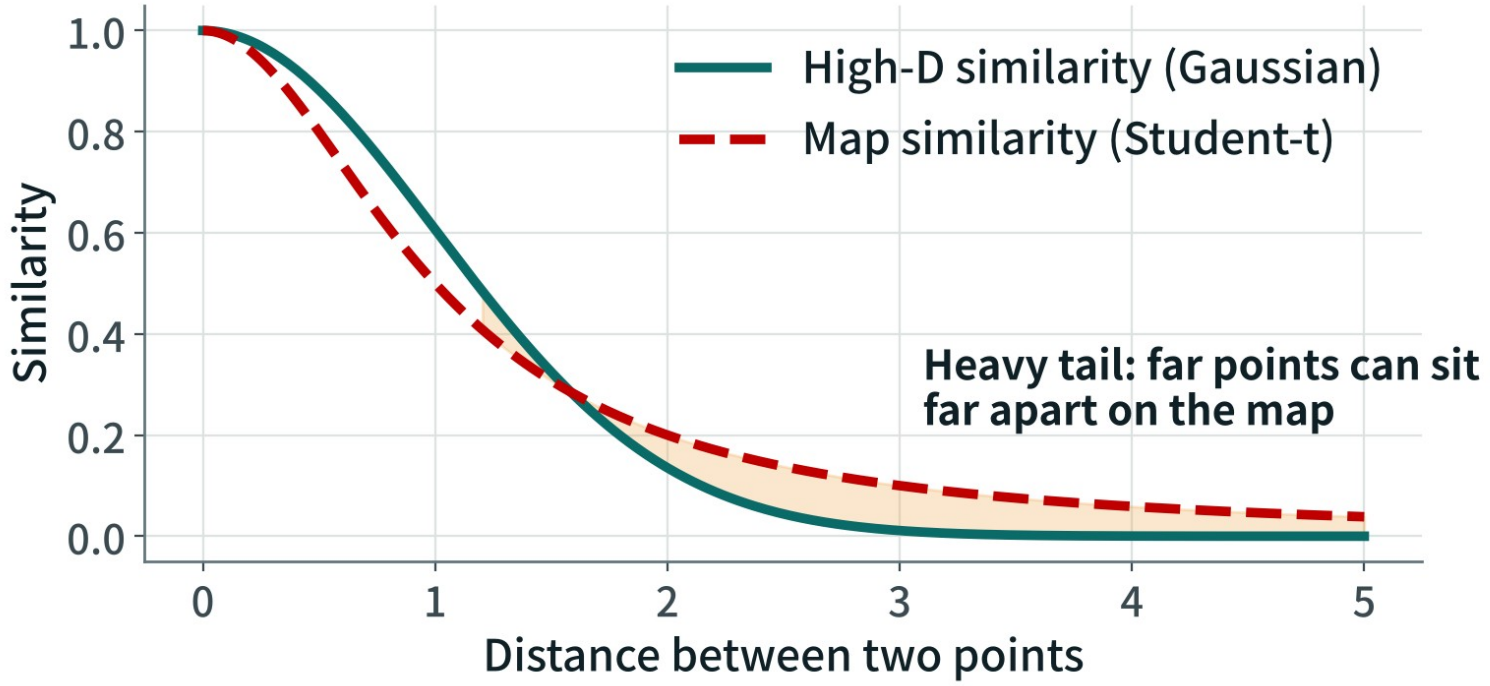
The heavy-tailed map kernel keeps clusters apart

Similarity as a function of distance in each space

Crowding problem

- 2-D has little room
- Gaussian crowds them
- Heavy tail frees them

Clean, separated islands

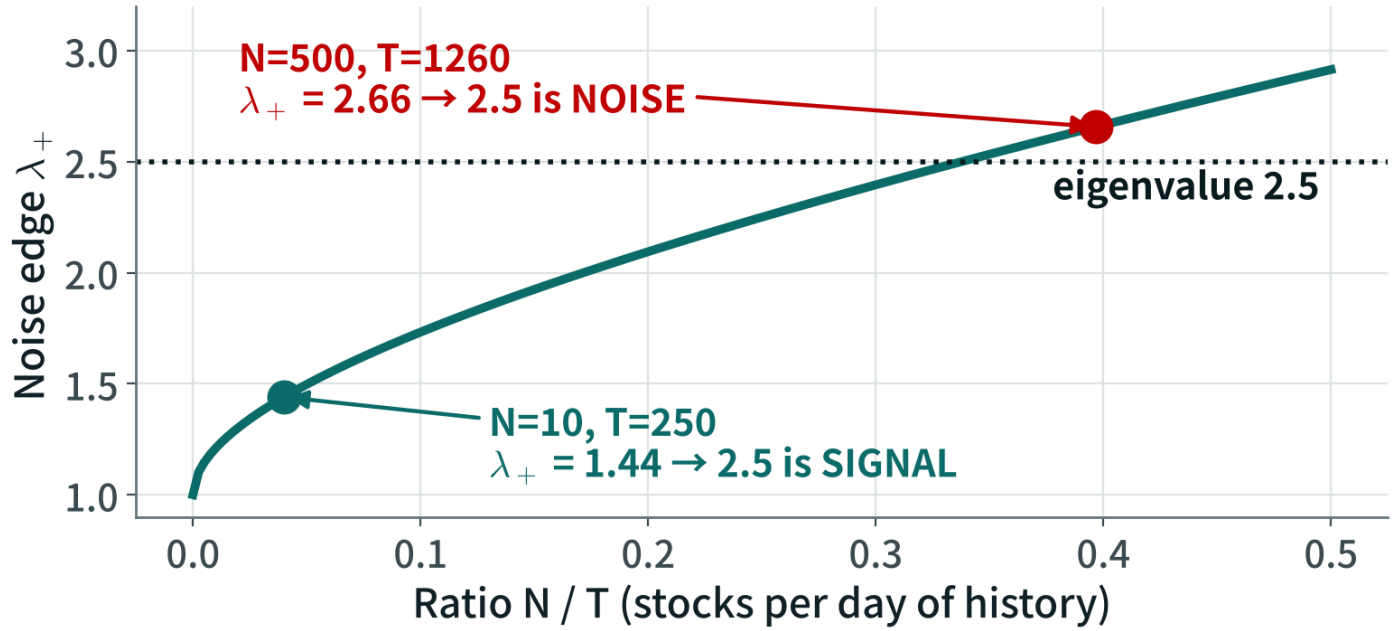


The same eigenvalue can be signal or noise

N = number of stocks, T = number of days. Eigenvalues below the edge are noise

Marchenko-Pastur noise edge

$$\lambda_+ = \left(1 + \sqrt{N/T}\right)^2$$



More stocks or less history raise the edge.
2.5 is signal for 10 stocks but noise for 500.

The minimum-variance portfolio puts the inverse covariance in charge

Global minimum-variance (GMV) portfolio: fully invested, lowest predicted risk (Markowitz)

GMV weights

$$w^* = \frac{\Sigma^{-1}\mathbf{1}}{\mathbf{1}^\top\Sigma^{-1}\mathbf{1}}$$

SETUP

50 stocks, 120 days

1 market + 5 sectors

RAW

Sample covariance

As estimated

DENOISED

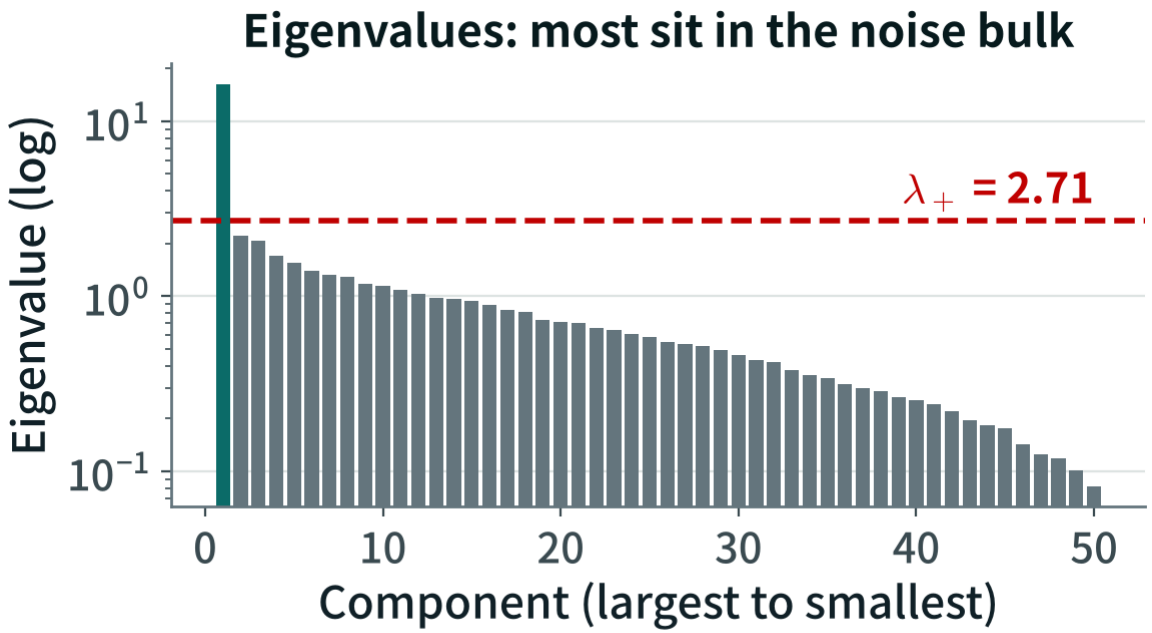
MP-cleaned

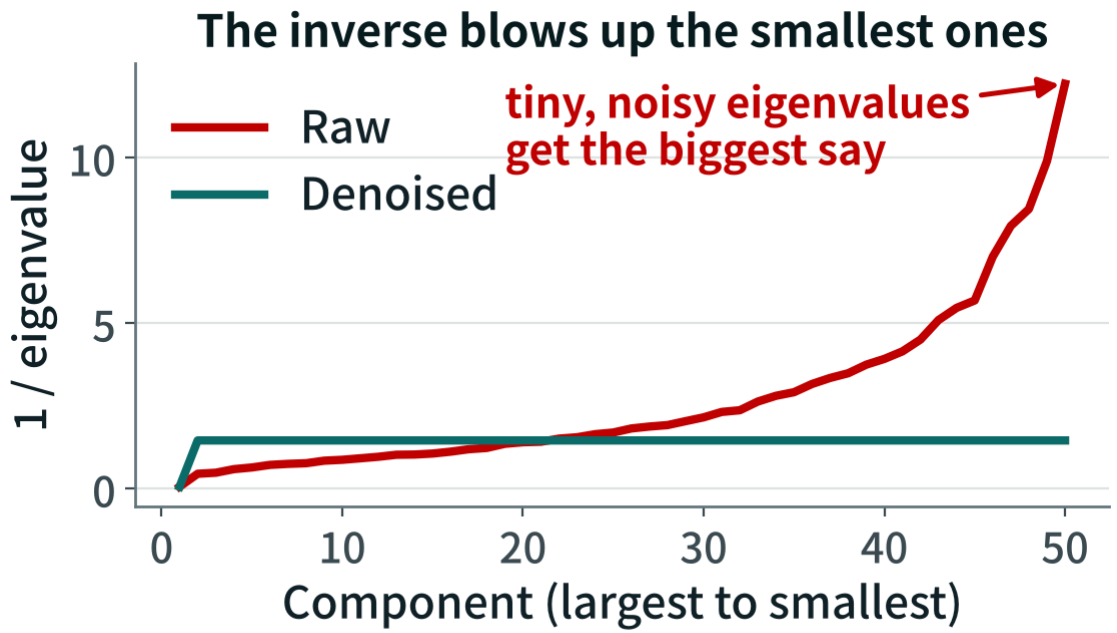
Same vols, clean corr

The inverse magnifies every estimation error in the matrix.

Inverting a noisy matrix turns tiny eigenvalues into big bets

Σ^{-1} divides by each eigenvalue, so the smallest, noisiest ones dominate the weights





※ Simulated: 50 stocks, 120 days, 1 market + 5 sector factors

※ Denoising sets the bulk to its average, so no noise direction gets an outsized say.

04

Build It with Claude Code

Four prompts reproduce every figure in this deck

Claude Code does the work while you check the result

Claude desktop app, Code tab. You paste a prompt and read the output

Open Code tab → **Paste prompt** → **Claude runs it** → **Read figures** → **Ask why**

CLAUDE DOES

Download, code, run

and fix its own errors

YOU DO

Paste and check

Does it make sense?

RULE

Explain every line

Ask why, not just do

Prompt 1 clusters the ten stocks three ways

Covers K-Means, the elbow, the dendrogram and DBSCAN

WHERE	INPUT	OUTPUT	FEEDBACK
Code tab	This prompt	4 PNG figures	Real sectors?

[Input] Prompt · Claude desktop app, Code tab

Download daily returns for AAPL, MSFT, GOOGL, AMZN, META, KO, PEP, XOM, JPM and JNJ from 2020 to 2024 with yfinance. Cluster the 10 stocks three ways: (1) K-Means with K from 1 to 8 and an elbow plot, (2) hierarchical clustering on correlation distance with average linkage and a dendrogram, (3) DBSCAN, choosing eps from a k-distance plot. Save every figure as PNG and open them for me. Finish with three sentences: do the groups match real sectors, and which stock is the odd one out? If anything fails, fix it yourself and rerun.

Prompt 2 compresses returns with PCA and maps digits with t-SNE

Covers the scree plot, PC1 loadings, PCA vs t-SNE and perplexity

WHERE	INPUT	OUTPUT	FEEDBACK
Code tab	This prompt	4 PNG figures	Market PC1?

[Input] Prompt · Claude desktop app, Code tab

Using the same 10-stock returns, standardize them and run PCA. Draw the scree plot with cumulative variance and a bar chart of PC1 loadings, and tell me whether PC1 behaves like a market factor. Then load scikit-learn's handwritten digits (0, 1, 4, 7 only) and draw PCA and t-SNE maps side by side, coloured by digit. Finally draw t-SNE at perplexity 3, 30 and 150. Save and open every figure, and explain each one in one plain sentence.

Prompt 3 denoises the correlation matrix before optimizing

Covers the eigenvalues, the Marchenko-Pastur edge and the before/after weights

WHERE	INPUT	OUTPUT	FEEDBACK
Code tab	This prompt	Table + 2 charts	Saner weights?

[Input] Prompt · Claude desktop app, Code tab

Compute the correlation matrix of the 10 stock returns and all its eigenvalues. Compute the Marchenko-Pastur edge using N stocks and T days, count how many eigenvalues are above it, replace the rest with their average, and rebuild the correlation matrix with ones on the diagonal. Plot the eigenvalues before and after with the edge as a dashed line. Print minimum-variance portfolio weights from the raw and the denoised matrix side by side, and explain in three sentences which weights look saner and why.

Prompt 4 replays every method one step at a time

Re-creates the step-by-step pictures of this deck as animations

WHERE	INPUT	OUTPUT	FEEDBACK
Code tab	This prompt	3 GIF files	Narrate it?

[Input] Prompt · Claude desktop app, Code tab

Make simulated 2-D data with three groups. Create three step-by-step animations and save each as a GIF: (1) K-Means with K = 3 from a poor start, one frame per assign and per update, showing the centers moving; (2) hierarchical clustering on six labelled points, one frame per merge, with the dendrogram growing next to the scatter; (3) DBSCAN with eps = 1 and min_samples = 4, first labelling core, border and noise points, then growing each cluster. Open the GIFs for me and explain each frame in one plain sentence.

GROUP THE ROWS.

SQUEEZE THE COLUMNS.

CLEAN THE MATRIX.

Structure found without labels
must still make economic sense.